1 INTRODUCTION

Anemia is a hematological condition characterised by a deficiency in the number or quality of red blood cells, resulting in reduced oxygen-carrying capacity of the blood. It remains one of the most prevalent blood disorders worldwide, affecting individuals across all age groups, and manifests in several distinct clinical subtypes depending on its underlying cause, ranging from nutritional deficiencies such as iron deficiency to more severe conditions such as leukemia-associated anemia. Timely and accurate identification of the specific anemia subtype is essential for effective clinical management, since the diagnostic and treatment pathway differs considerably between subtypes.

A Complete Blood Count (CBC) test is a routine, low-cost, and widely available laboratory investigation that reports a panel of hematological parameters, including white blood cell count, red blood cell indices, hemoglobin concentration, platelet count, and related derived measures. Because these parameters vary in characteristic ways across different anemia subtypes, CBC data provides a rich and clinically meaningful basis for automated classification using machine learning techniques. However, manual interpretation across high patient volumes can be labor-intensive, subject to inter-observer variability, and often restricted to coarse-grained or binary screening that fails to isolate specific underlying etiologies.

Despite the wide availability of CBC data, several challenges limit its use for subtype-level diagnosis. Reading the pattern across many parameters at once, for example how hemoglobin, red cell size and hemoglobin concentration change together, depends on the experience of the person reviewing the report. Most published machine learning studies reduce the problem to anemic versus non-anemic, which does not tell a clinician which condition to investigate or treat. In addition, clinical hematology data is naturally imbalanced: in the dataset used here the largest class has 336 records and the smallest only 11, a ratio of about 30 to 1, so a model trained without care tends to favour the common classes and overlook the rare but serious ones. Finally, few studies turn their models into a tool that a laboratory worker can actually use.

This project, titled “Anemia Detection and Classification using CBC Blood Test Data”, proposes a machine learning based clinical decision-support system that classifies a patient’s complete blood count into one of nine categories: eight distinct anemia subtypes and a no-anemia (healthy) category. The system uses Random Forest as the primary classification algorithm, selected for its ability to model complex, non-linear relationships across tabular hematological features, with XGBoost as a boosting benchmark and Logistic Regression as an interpretable baseline. To resolve the severe class imbalance inherent in real-world clinical datasets where rare conditions are significantly underrepresented, the Synthetic Minority Oversampling Technique (SMOTE) is applied specifically to the training partition to protect minority-class sensitivity without introducing test-set leakage.

The objectives of the project are therefore to build a multi-class classifier that maps 14 routine CBC parameters to one of nine diagnostic classes; to compare Random Forest, XGBoost and Logistic Regression on an identical stratified 80/20 split so that their results are directly comparable; to handle class imbalance with SMOTE applied to the training data only, keeping the test set untouched; to evaluate the models using accuracy, precision, recall, F1-score, confusion matrices and ROC curves rather than accuracy alone; and to deploy the best model in a Streamlit application that accepts values entered manually or uploaded as a CSV or Excel file.

The trained model is implemented through an interactive Streamlit web application featuring a custom-styled interface, allowing users to input complete blood count values and receive real-time diagnostic predictions. By executing real-time inference on 14 standard CBC inputs, the platform serves as an accessible, assistive screening aid designed to accelerate diagnostic turnaround and support medical personnel in resource-constrained environments. This report documents the complete project, detailing the supporting literature and identified research gaps, dataset exploration and preprocessing strategies, algorithmic design, end-to-end pipeline implementation, and a comprehensive feasibility analysis of the proposed clinical system.

The system is intended as a preliminary screening aid that supports, and does not replace, clinical judgment. It was developed and tested on a single public dataset of 1,281 records, and the rarest classes have very few test samples, so the reported results should be read as a demonstration of feasibility rather than as clinical validation. The report is organised as follows. Chapter 2 reviews the supporting literature and the research gap. Chapter 3 analyses the dataset, preprocessing, algorithms, pipeline and feasibility, and Chapter 4 describes model building. Chapter 5 presents and discusses the results, and Chapter 6 covers deployment as a web application. Chapter 7 gives the Git history, and Chapters 8 and 9 present the conclusions and future scope. The appendix lists the software and hardware requirements, followed by the references.

2 SUPPORTING LITERATURE

2.1 Literature Review

Paper 1: M. S. Farooq, M. Hassan, G. Muhammad, O. Ali, Z. Zeeshan, M. Saleem, M. Ahmad, S. Abbas, M. A. Khan, and T. M. Ghazal. "Developing a Transparent Anaemia Prediction Model Empowered With Explainable Artificial Intelligence." *IEEE Access*, vol. 13, pp. 1307–1318, 2025.

Machine learning models for anaemia prediction often act as black boxes, limiting clinicians' trust despite their high accuracy. The authors addressed this by proposing a prediction model that utilizes post-hoc interpretability methods to interpret feature contributions in blood test data.

Although the study achieves high predictive accuracy (98.13% using Support Vector Machines), it is restricted to general binary anaemia detection and does not differentiate between underlying anaemia subtypes. This identifies a clear research gap for an end-to-end multi-class classification system capable of isolating specific disease subtypes directly from routine Complete Blood Count parameters, which forms the primary focus of this project.

Category	Details
Title	Developing a Transparent Anaemia Prediction Model Empowered With Explainable Artificial Intelligence
Area of Work	Automated anemia prediction using machine learning classifiers
Dataset	Clinical CBC dataset containing routine patient blood parameters
Methodology / Strategy	Data preprocessing, feature evaluation, train-test split, model training, and performance evaluation
Algorithm(s)	SVM, Decision Tree, KNN, Gradient Boosting
Result / Precision	SVM achieved 98.13% classification accuracy
Advantages	Routine CBC attributes provide sufficient discriminative power for automated anemia screening
Future Proposal	Extend the model for multi-class classification of specific anemia subtypes

Paper 2: D. C. E. Saputra, K. Sunat, and T. Ratnaningsih. "SMOTE-MRS: A Novel SMOTE-Multiresolution Sampling Technique for Imbalanced Distribution to Improve Prediction of Anemia." *IEEE Access*, vol. 12, pp. 154675–154699, 2024.

Hematological datasets are frequently highly imbalanced, causing standard machine learning algorithms to develop bias toward majority classes and perform poorly on rare but critical clinical conditions. In such datasets, overall accuracy can look high even when most minority-class patients are missed, which is a serious concern in medical screening. To address this, the authors proposed SMOTE-MRS, a hybrid oversampling technique combining SMOTE, K-Means clustering, and random oversampling to balance the class distribution. This is directly relevant to the present project, where the rarest classes contain only 11 and 18 records in total.

Their evaluation demonstrated that handling class imbalance drastically improves minority-class recall and F1-scores. However, the paper focused exclusively on the resampling technique itself rather than an end-to-end clinical diagnostic system. This directly informs this project's decision to integrate SMOTE-based class balancing into an end-to-end multi-class classification pipeline.

Category	Details
Title	SMOTE-MRS: A Novel SMOTE-Multiresolution Sampling Technique for Imbalanced Distribution to Improve Prediction of Anemia
Area of Work	Handling severe class imbalance in clinical hematology datasets using synthetic oversampling.
Dataset	Imbalanced clinical anemia datasets containing routine hematological measures.
Methodology / Strategy	Data preprocessing, cluster-based synthetic oversampling, classifier training, and minority-class recall evaluation.
Algorithm(s)	SMOTE-MRS combined with classical tree-based classifiers.
Result / Precision	Significantly improved F1-score and Recall for minority classes compared to the raw imbalanced dataset.
Advantages	Prevents majority-class bias and improves diagnostic sensitivity for rare blood disorders.
Future Proposal	Recommended integrating hybrid oversampling methods into complete multi-class anemia diagnostic systems.

Paper 3: M. M. Rahman, M. U. Mojumdar, H. A. Shifa, N. R. Chakraborty, S. N. P. Stenin, and M. A. Hasan. "Anemia Disease Prediction Using Machine Learning Techniques and Performance Analysis." *Proc. 2024 11th Int. Conf. Computing for Sustainable Global Development (INDIACom)*, pp. 1276–1282, 2024.

Manual interpretation of blood test reports is time-consuming, and automated tools reported in literature often evaluate small datasets or simple tasks. The authors evaluated multiple classical machine learning models on one thousand patient records using eight routine CBC features. The compared models were Logistic Regression, Decision Tree, Random Forest and Support Vector Machine, trained after data preprocessing and feature correlation analysis using an 80:20 train-test split. Logistic Regression achieved 95% classification accuracy, indicating that routine CBC parameters carry strong discriminative signal. However, the study was bounded by a binary target (0 vs 1), leaving unaddressed the challenge of isolating specific anemia subtypes. Because the dataset was small and the task binary, the reported accuracy shows that CBC values are informative for detecting anemia, but it does not show that subtype-level diagnosis is achievable. The authors also noted the limited size of their dataset and recommended increasing it and extending the framework to multi-class subtype classification. This project follows that recommendation by classifying 1,281 records into nine diagnostic classes using 14 CBC parameters.

Category	Details
Title	Anemia Disease Prediction Using Machine Learning Techniques and Performance Analysis
Area of Work	Comparative evaluation of machine learning models for automated anemia detection using blood test data.
Dataset	1,000 patient records containing 8 CBC features.
Methodology / Strategy	Data preprocessing, feature correlation analysis, 80:20 train-test split, model training, and comparative evaluation.
Algorithm(s)	Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM).
Result / Precision	Logistic Regression achieved 95% classification accuracy.
Advantages	Demonstrated high diagnostic accuracy using routine, low-cost CBC parameters.
Future Proposal	Recommended increasing dataset size and extending the framework to multi-class anemia subtype classification.

2.2 Literature Review Summary

Author(s) & Year	Problem Focus	Dataset Size / Scope	Algorithms Evaluated	Primary Limitation
M. S. Farooq, M. Hassan, G. Muhammad, O. Ali, Z. Zeeshan, M. Saleem, M. Ahmad, S. Abbas, M. A. Khan, T. M. Ghazal (2025)	Binary Anemia Prediction	Clinical CBC data	SVM, Decision Tree, KNN, Gradient Boosting	General binary anemia prediction only; no subtype classification.
M. M. Rahman, M. U. Mojumdar, H. A. Shifa, N. R. Chakraborty, S. N. P. Stenin, and M. A. Hasan (2024)	Performance analysis of ML classifiers for anemia	1,000 patient records, 8 attributes	Logistic Regression, Decision Tree, Random Forest, SVM	Limited to binary classification (anemic vs. non-anemic).
D. C. E. Saputra, K. Sunat, and T. Ratnaningsih (2024)	Imbalanced class resampling for anemia	Imbalanced clinical hematology datasets	SMOTE-MRS + Tree Classifiers	Focused on the resampling algorithm itself, not an end-to-end pipeline.

Table 2.1: Summary of Existing Research on Machine Learning-Based Anemia Detection

2.3 Findings and Proposals

Manual interpretation of Complete Blood Count (CBC) reports to identify the specific subtype of anemia is labor-intensive, time-consuming, and heavily dependent on the clinical expertise and subjective judgment of examining pathologists or laboratory technicians. In high-volume diagnostic centers and primary healthcare facilities, this manual evaluation leads to operational bottlenecks and inter-observer diagnostic inconsistency. Furthermore, existing automated computational screening models in the literature predominantly frame anemia diagnosis as a simple binary task (anemic versus non-anemic) or a coarse two-class differential. In a real-world clinical setting, a generic "anemic" flag is inadequate for prescribing immediate, targeted interventions, as clinical management differs drastically depending on whether the underlying etiology stems from iron deficiency, marrow failure, or a malignant hematological condition.

The three studies reviewed in this chapter each address one part of the problem and leave another open. Farooq et al. and Rahman et al. show that routine CBC parameters are enough for accurate anemia detection, reporting accuracies of 98.13% (SVM) and 95% (Logistic Regression), but both treat the task as binary. Saputra et al. show that correcting class imbalance improves minority-class recall and F1-score, but they study the resampling method rather than a complete diagnostic system. Taken together, the literature supports the view that CBC data is informative

and that imbalance must be handled, yet no reviewed study combines subtype-level classification, careful imbalance handling and a usable interface in one pipeline. This is the gap the present project is designed to fill.

While extensive research has demonstrated high benchmark accuracy for binary anemia classification from routine blood parameters, there remains a critical research and translation gap for fine-grained, multi-class diagnostic frameworks capable of categorizing comprehensive CBC profiles into specific pathological subtypes. Additionally, real-world clinical hematology data suffers from severe, natural class imbalance; common conditions and healthy controls heavily outnumber rare, life-threatening subtypes such as Macrocytic Anemia or Leukemia with Thrombocytopenia. Most existing studies fail to implement rigorous, isolated resampling methodologies, resulting in models that exhibit strong majority-class bias and critically low diagnostic sensitivity for rare but urgent hematological disorders. Finally, there is a distinct lack of accessible, lightweight clinical decision-support tools that bridge this algorithmic capability directly into an end-user web interface suitable for resource-constrained laboratories.

To address these gaps, this project proposes an end-to-end, automated Clinical Decision Support System (CDSS) that performs fine-grained 9-class diagnostic classification (one healthy control group alongside eight distinct anemia and hematological subtypes) directly from fourteen routine CBC parameters. The core classification engine is built upon an ensemble Random Forest architecture, benchmarked in parallel against XGBoost and an interpretable Logistic Regression baseline.

Several design decisions follow from these findings. The target is a nine-class problem, with a healthy class alongside eight anemia and related subtypes, so that the output names a specific condition instead of a generic “anemic” flag. All 14 CBC parameters are retained. A correlation analysis that excluded out-of-range and mean-imputed values found four strongly correlated pairs (HGB–HCT, PLT–PCT, LYMp–NEUTp and WBC–NEUTn), which is expected from their physiological definitions, and tree-based models are not adversely affected by correlated inputs. Three models are compared on the same split: Random Forest as the primary model, XGBoost as a boosting benchmark, and Logistic Regression as a linear baseline that shows how much the non-linear models add. Using one stratified 80/20 split with a fixed random seed keeps the comparison fair and reproducible.

To reduce minority-class bias, class imbalance is handled with SMOTE applied to the training partition only. The largest class (Healthy) is about 30 times the size of the smallest (Leukemia with thrombocytopenia), so SMOTE raises every class in the training set to 269 samples, growing it from 1,024 to 2,421 records, while the 257-sample test set keeps its original class proportions. Keeping the test set untouched means the reported scores are not inflated by synthetic samples and no data leaks into the evaluation phase. The trade-off is that the rarest classes contribute very few test cases (4 for Macrocytic anemia and 2 for Leukemia with thrombocytopenia), so

results on those classes are indicative rather than conclusive.

The experiments show a clear ordering of the models. Random Forest reached 98.83% accuracy and a weighted F1-score of 98.89%, misclassifying 3 of 257 test samples. XGBoost reached 96.89% accuracy but scored an F1 of 0.00 on Leukemia with thrombocytopenia, a result based on only two test samples. Logistic Regression reached 74.32%, about 24.5 percentage points below Random Forest, which indicates that the boundaries between diagnostic classes in CBC space are non-linear. Both ensemble methods therefore suit this problem far better than a linear model, with Random Forest the strongest overall.

The features that drive the predictions are clinically sensible, which supports the validity of the approach. MCV, hemoglobin and MCH ranked highest, and these are the standard indicators used to classify anemia by hemoglobin level and red cell size and colour. The class medians behave as expected: HGB is 13.9 g/dL in the Healthy class but 9.8 g/dL in Iron deficiency anemia, median WBC is highest in Leukemia (12.8), and median platelet count falls to 120 in Thrombocytopenia compared with 330 in the Healthy class. That the model relies on the same signals a clinician would use makes its predictions easier to trust and to explain.

The trained Random Forest is deployed as a lightweight Streamlit web application. Clinicians and laboratory technicians can enter the 14 routine CBC values manually or upload them as a CSV or Excel file, and receive an instant predicted class together with the class probabilities and a plain-language explanation.

These findings come with limitations that shape the proposals for further work. The dataset is small (1,281 records) and comes from a single public source. The data-quality checks found 49 duplicate rows and 38 values outside plausible physiological ranges, and the inspection of the raw file suggests that missing values in several columns were replaced by the column mean before publication. These records were identified but not removed before the split, so the reported accuracy may be somewhat optimistic. In the application, the probability of Leukemia with thrombocytopenia is multiplied by a decision weight of 2.0 so that this rare, serious class is not overlooked; on the test set this changed none of the 257 predictions (Table 5.6). The natural next steps are therefore to repeat the experiments on cleaned data with cross-validation, and to validate the system on larger, independent datasets before any clinical use.

3 SYSTEM ANALYSIS

Currently, anemia diagnosis relies on manual interpretation of CBC reports by clinicians, who compare hemoglobin, RBC, hematocrit and other parameters against standard reference ranges. This process is time-consuming, prone to human error and dependent on the individual's clinical experience. It scales poorly in high-patient-volume settings and offers no automated way to flag anemia subtypes early.

A further limitation is that reference ranges are applied one parameter at a time. Subtype differences often appear as a combined pattern, for example low hemoglobin together with small red cells and low hemoglobin concentration in iron deficiency, or an abnormal white cell count together with low platelets in leukemia with thrombocytopenia. Spotting such combinations across 14 values is difficult to do consistently by eye, and the result can differ from one reviewer to another. Existing automated screening tools reported in the literature mostly stop at an anemic or non-anemic flag, which leaves the clinician to work out the underlying condition.

The proposed system addresses these limitations by using classical machine learning algorithms (Logistic Regression, Random Forest and XGBoost) trained on the 14 routine CBC parameters (WBC, LYMp, NEUTp, LYMn, NEUTn, RBC, HGB, HCT, MCV, MCH, MCHC, PLT, PDW and PCT). Instead of a binary flag, it classifies each patient's CBC profile into one of nine diagnostic classes, offering a faster, more consistent and scalable decision-support tool that reduces subjectivity in the initial screening process. It requires only standard computing resources and open-source Python libraries, which makes it technically and economically feasible for practical use.

The proposed system takes the 14 routine CBC parameters as input and returns one of nine diagnostic classes (a healthy class and eight anemia and related hematological subtypes) together with the class probabilities. Values can be entered manually or uploaded as a CSV or Excel file for batch prediction. The system is intended as a preliminary screening aid to support, not replace, clinical judgment.

The system is required to perform the following functions:

- Accept the 14 CBC parameters for one patient through a form, with each field limited to a plausible physiological range.
- Accept a CSV or Excel file with one row per patient, match its column names regardless of letter case, and predict every valid row.
- Return the predicted diagnostic class together with the top class probabilities and a plain-language explanation of the predicted class.
- Warn the user when the entered hematocrit is inconsistent with the RBC and MCV values, and when the two most probable classes are nearly tied.

- Skip rows with missing, non-numeric or out-of-range values during batch prediction, record a note for each, and allow the results to be downloaded as a CSV file.
- Provide reference material, namely the normal ranges of the 14 parameters and a description of each of the nine classes.

Beyond its functions, the system must meet the following qualities:

- *Speed:* predictions should be returned in real time, which is achieved by loading a saved model file instead of retraining at start-up.
- *Usability:* the interface should be usable by a laboratory technician or clinician without machine learning knowledge.
- *Reproducibility:* a fixed random seed is used for the data split, SMOTE and all three models, so the experiments can be repeated with the same results.
- *Portability:* the application should run in any modern web browser, with only open-source libraries and no specialised hardware.
- *Safety:* the system must present itself as a screening aid, never as a replacement for clinical judgment, and must signal uncertain predictions instead of hiding them.

The system is built on a single public dataset of 1,281 records, so its behaviour on data from other laboratories, analyzers or patient populations is unknown. It assumes that the input values are entered in the units used by the dataset, which the interface displays beside each field. The rarest classes have very few examples, so predictions for them are less reliable than for the common classes. These constraints are discussed further in the results and conclusions.

3.1 Analysis of the Dataset

3.1.1 About the Dataset

The Anemia Types Classification dataset is obtained from Kaggle, contributed by Ehab Aboelnaga. It contains 1,281 patient records structured across 15 columns, consisting of 14 continuous hematological parameters and one categorical target attribute (Diagnosis). The target variable encompasses nine distinct diagnostic classes: one healthy non-anemic control class and eight specific blood disorder subtypes, namely Normocytic Hypochromic Anemia, Normocytic Normochromic Anemia, Iron Deficiency Anemia, Thrombocytopenia, Other Microcytic Anemia, Leukemia, Macrocytic Anemia, and Leukemia with Thrombocytopenia. The data is distributed in a standard tabular CSV format (`diagnosed_cbc_data_v4.csv`), where each row corresponds to a single patient's Complete Blood Count (CBC) profile, allowing direct ingestion without structural format restructuring.

For this project's machine learning classification pipeline, all fourteen hematological parameters were retained for model development, without feature-count reduction. This decision is supported by the correlation and feature-importance analysis in Section 3.3, which shows that correlated features are naturally handled by the tree-based ensemble methods without requiring explicit dimensionality reduction.

- Dataset Source: Kaggle – Anemia Types Classification Dataset, contributed by Ehab Aboelnaga.
- Raw File Used: `diagnosed_cbc_data_v4.csv` (tabular CSV format).
- Patient Records: 1,281 total raw entries.
- Diagnosis Categories (Classes): 9 distinct clinical outcomes (1 Healthy control + 8 Anemia subtypes).
- CBC Features per Record: 14 continuous parameters available, all retained for model training.

The 14 parameters fall into three groups: white blood cell measures (WBC, LYMp, NEUTp, LYMn, NEUTn), red blood cell measures and indices (RBC, HGB, HCT, MCV, MCH, MCHC), and platelet measures (PLT, PDW, PCT). The classes are unevenly represented: Healthy has 336 records, Normocytic hypochromic anemia 279, Normocytic normochromic anemia 269, Iron deficiency anemia 189, Thrombocytopenia 73, Other microcytic anemia 59, Leukemia 47, Macrocytic anemia 18, and Leukemia with thrombocytopenia 11.

The class distribution of the dataset is markedly imbalanced, which is an important characteristic for both model training and evaluation. The Healthy control class is the largest at 336 records (26.2% of the dataset), followed by Normocytic hypochromic anemia with 279 records (21.8%), Normocytic normochromic anemia with 269 records (21.0%), and Iron deficiency anemia with 189 records (14.8%). Together, these four classes account for roughly 84% of all samples. In contrast, Thrombocytopenia (73 records, 5.7%), Other microcytic anemia (59 records, 4.6%), Leukemia (47 records, 3.7%), Macrocytic anemia (18 records, 1.4%), and Leukemia with thrombocytopenia (11 records, 0.9%) form a sparsely represented minority group. The ratio between the largest and smallest classes is approximately 30.5:1, so a classifier trained on the raw distribution would be biased toward the majority classes and could achieve a deceptively high overall accuracy while failing to recognise rare conditions. This is clinically undesirable, because the rare classes, such as leukemia and its combination with thrombocytopenia, are often the most serious diagnoses. This imbalance motivated three design choices in the pipeline: a stratified train–test split, so that every class keeps the same proportion in both partitions; oversampling of the minority classes using SMOTE applied to the training set only, so that no synthetic samples leak into the test set; and evaluation with per-class precision, recall, and F1-score, together with confusion matrices and ROC curves, instead of overall accuracy alone.

Clinically, the 14 parameters are complementary. The red blood cell indices (MCV, MCH, and MCHC) describe the size and hemoglobin content of red cells and are the traditional basis for morphological anemia classification into microcytic, normocytic, and macrocytic types, while HGB, HCT, and RBC indicate the severity of anemia. The platelet measures (PLT, PDW, and PCT) are decisive for thrombocytopenia-related classes, and the white blood cell counts and differentials (WBC, LYMp, NEUTp, LYMn, and NEUTn) help identify leukemia-related patterns. Because the values are standard laboratory outputs routinely produced by automated hematology analyzers, the dataset reflects the kind of information available in everyday clinical practice, which makes the resulting classifier practical as a screening and decision-support aid. A preliminary inspection of the file confirmed that no missing values were present in any of the 15 columns.

Feature	Full form	Unit	Male range	Female range
WBC	White blood cell count	$\times 10^3/\mu$L	4.0–11.0	4.0–11.0
LYMp	Lymphocyte percentage	%	20–40	20–40
NEUTp	Neutrophil percentage	%	40–70	40–70
LYMn	Lymphocyte number (absolute count)	$\times 10^3/\mu$L	1.0–4.8	1.0–4.8
NEUTn	Neutrophil number (absolute count)	$\times 10^3/\mu$L	1.8–7.7	1.8–7.7
RBC	Red blood cell count	$\times 10^6/\mu$L	4.5–5.9	4.1–5.1
HGB	Hemoglobin	g/dL	13.5–17.5	12.0–15.5
HCT	Hematocrit	%	41–53	36–46
MCV	Mean corpuscular volume	fL	80–100	80–100
MCH	Mean corpuscular hemoglobin	pg	27–33	27–33
MCHC	Mean corpuscular hemoglobin concentration	g/dL	32–36	32–36
PLT	Platelet count	$\times 10^3/\mu$L	150–450	150–450
PDW	Platelet distribution width	fL	9–17	9–17
PCT	Plateletcrit	%	0.15–0.40	0.15–0.40

Table 3.1: CBC parameters, units and typical adult reference ranges

Table 3.1 lists the 14 CBC parameters together with their units and typical adult reference ranges. These ranges are indicative only, since exact limits vary slightly between laboratories, analyzers, and populations. Most parameters share a single range for both sexes, but the red

cell measures (RBC, HGB, and HCT) have lower reference limits for females, mainly because of menstrual blood loss and hormonal differences. The dataset contains no age or sex column, so each record must be interpreted without demographic context. This is a limitation, as a hemoglobin value that is normal for a woman may be mildly low for a man, and the models must learn the distinguishing patterns from the CBC values alone. The percentage and absolute count pairs (LYMp with LYMn, and NEUTp with NEUTn) are closely related, because each absolute count is approximately the product of the total WBC and the corresponding percentage.

Class	Description and typical CBC pattern
Healthy	All parameters within normal ranges; no sign of anemia or a blood disorder
Iron deficiency anemia	Too little iron to make hemoglobin; small, pale red cells (low HGB, MCV, MCH and MCHC)
Other microcytic anemia	Small red cells (low MCV) from causes other than simple iron deficiency
Macrocytic anemia	Enlarged red cells (high MCV) with low HGB and RBC
Normocytic normochromic anemia	Normal-sized, normally coloured red cells but too few of them (low HGB and RBC)
Normocytic hypochromic anemia	Normal-sized but pale red cells (low MCHC) with low HGB
Leukemia	Abnormal white cells; WBC usually high; a blood count can only suggest it
Leukemia with thrombocytopenia	Leukemia pattern together with a low platelet count (PLT below 150)
Thrombocytopenia	Low platelet count (below 150 $\times 10^3/\mu$L); red cell values usually normal

Table 3.2: The nine diagnostic classes

Table 3.2 summarises the nine diagnostic classes and the CBC pattern that characterises each. The classes can be grouped by the cell line they mainly affect: the red cell disorders (Iron deficiency, Other microcytic, Macrocytic, and the two Normocytic anemias), the platelet disorder (Thrombocytopenia), the white cell disorders (Leukemia), and the combined condition (Leukemia with thrombocytopenia). Some classes are separated by only a small difference in a single index. For example, the two normocytic anemias differ mainly in MCHC, and Iron deficiency anemia and Other microcytic anemia both present with a low MCV, so their boundaries overlap in feature space. These overlaps, together with the small sample sizes of

the rare classes, make the nine-class problem considerably harder than a binary anemia versus healthy decision. They also explain why the evaluation in later chapters reports per-class metrics rather than a single overall score. It should also be emphasised that a blood count alone cannot confirm a diagnosis such as leukemia; the classifier is therefore intended as a screening and decision-support aid, and its predictions should be confirmed by a clinician through further tests.

Class	Records	Share
Healthy	336	26.2%
Normocytic hypochromic anemia	279	21.8%
Normocytic normochromic anemia	269	21.0%
Iron deficiency anemia	189	14.8%
Thrombocytopenia	73	5.7%
Other microcytic anemia	59	4.6%
Leukemia	47	3.7%
Macrocytic anemia	18	1.4%
Leukemia with thrombocytopenia	11	0.9%
Total	1,281	100%

Table 3.3: Number and share of records in each diagnostic class

3.1.2 Explore the Dataset

a) Shape and data types

```
df = pd.read_csv('diagnosed_cbc_data_v4.csv')
print('shape :', df.shape)
print(df.dtypes)
```

```
shape : (1281, 15)
WBC           float64
LYMp          float64
NEUTp         float64
LYMn          float64
NEUTn         float64
RBC           float64
HGB           float64
HCT           float64
MCV           float64
MCH           float64
MCHC          float64
PLT           float64
PDW           float64
PCT           float64
Diagnosis      object
dtype: object
```

Figure 3.1: Shape and data types of the flattened anemia dataset

b) First five records

	WBC	LYMp	NEUTp	LYMn	NEUTn	RBC	HGB	HCT	MCV	MCH	MCHC	PLT	PDW	PCT	Diagnosis
0	10.0	43.2	50.1	4.3	5.0	2.77	7.3	24.2	87.7	26.3	30.1	189.0	12.5	0.17	Normocytic hypochromic anemia
1	10.0	42.4	52.3	4.2	5.3	2.84	7.3	25.0	88.2	25.7	20.2	180.0	12.5	0.16	Normocytic hypochromic anemia
2	7.2	30.7	60.7	2.2	4.4	3.97	9.0	30.5	77.0	22.6	29.5	148.0	14.3	0.14	Iron deficiency anemia
3	6.0	30.2	63.5	1.8	3.8	4.22	3.8	32.8	77.9	23.2	29.8	143.0	11.3	0.12	Iron deficiency anemia
4	4.2	39.1	53.7	1.6	2.3	3.93	0.4	316.0	80.6	23.9	29.7	236.0	12.8	0.22	Normocytic hypochromic anemia

Figure 3.2: First five records

c) Missing Values

```
print('MISSING VALUES:')
print(df.isnull().sum())
```

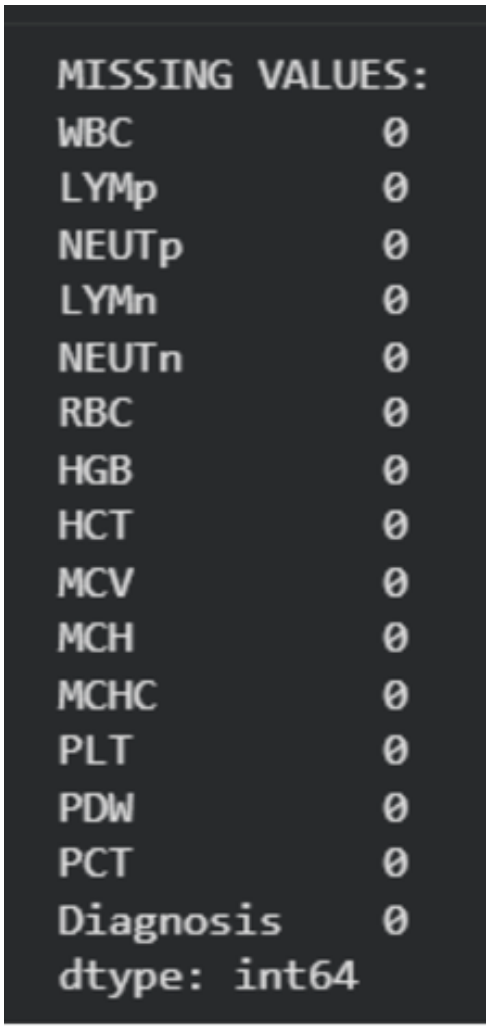


Figure 3.3: Missing Values

d) Summary statistics

```
import pandas as pd

# Load dataset
df = pd.read_csv('/content/drive/MyDrive/dataset_anemia/diagnosed_cbc_data_v4.
    csv')

# Summary statistics
print('SUMMARY STATISTICS:')
summary = df.describe()
print(summary)
```

```
SUMMARY STATISTICS:
                WBC         LYMp         NEUTp         LYMn         NEUTn  \
count  1281.000000  1281.000000  1281.000000  1281.000000  1281.000000
mean      7.862717    25.845000    77.511000     1.880760     5.140940
std       3.564466     7.038728   147.746273     1.335689     2.872294
min       0.800000     6.200000     0.700000     0.200000     0.500000
25%       6.000000    25.845000    71.100000     1.880760     5.100000
50%       7.400000    25.845000    77.511000     1.880760     5.140940
75%       8.680000    25.845000    77.511000     1.880760     5.140940
max      45.700000    91.400000  5317.000000    41.800000    79.000000

                RBC          HGB          HCT          MCV          MCH  \
count  1281.000000  1281.000000  1281.0000  1281.000000  1281.000000
mean      4.708267    12.184551    46.1526    85.793919    32.084840
std       2.817200     3.812897   104.8861    27.177663   111.170756
min       1.360000   -10.000000     2.0000   -79.300000    10.900000
25%       4.190000    10.800000    39.2000    81.200000    25.500000
50%       4.600000    12.300000    46.1526    86.600000    27.800000
75%       5.100000    13.500000    46.1526    90.200000    29.600000
max      90.800000    87.100000  3715.0000   990.000000  3117.000000

               MCHC          PLT          PDW          PCT
count  1281.000000  1281.000000  1281.000000  1281.000000
mean     31.739149   229.981421    14.312512     0.260280
std       3.300352    93.019336     3.005079     0.685351
min      11.500000    10.000000     8.400000     0.010000
25%      30.600000   157.000000    13.300000     0.170000
50%      32.000000   213.000000    14.312512     0.260280
75%      32.900000   293.000000    14.700000     0.260280
max      92.800000   660.000000    97.000000    13.600000
```

Figure 3.4: Summary statistics of the 14 CBC features

The `describe()` output reports the count, mean, standard deviation, minimum, quartiles, and maximum for each of the 14 numerical features, and gives a first quantitative view of how the data are distributed. The count row shows that every feature contains the same number of observations, which agrees with the absence of missing values. The mean and median (50% quartile) of each feature were compared to judge symmetry: a mean noticeably larger than the median suggests a right-skewed distribution, as is typical for count-based parameters such as WBC, LYMn, NEUTn, and PLT, where a few patients with extreme values pull the average upward. The standard deviation shows that the features differ widely in scale. PLT is measured in the hundreds, while indices such as MCHC and PCT lie in much smaller ranges, which is why feature scaling is applied for the distance-sensitive Logistic Regression model but is unnecessary for the tree-based models. The minimum and maximum rows were checked against the reference ranges in Table 3.1, because values lying far outside any physiologically possible limit point to recording or entry errors rather than genuine clinical findings. Such records are handled in the preprocessing stage described in Section 3.2.

e) Dataset structure (info)

```
# Dataset structure and data types
print('DATASET INFORMATION:')
df.info()
```

The `info()` summary confirms that all 14 features are stored as 64-bit floating-point numbers, that the Diagnosis column is a text (object) column, and that every column has 1,281 non-null entries.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1281 entries, 0 to 1280
Data columns (total 15 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   WBC        1281 non-null   float64
 1   LYMp       1281 non-null   float64
 2   NEUTp      1281 non-null   float64
 3   LYMn       1281 non-null   float64
 4   NEUTn      1281 non-null   float64
 5   RBC        1281 non-null   float64
 6   HGB        1281 non-null   float64
 7   HCT        1281 non-null   float64
 8   MCV        1281 non-null   float64
 9   MCH        1281 non-null   float64
 10  MCHC       1281 non-null   float64
 11  PLT        1281 non-null   float64
 12  PDW        1281 non-null   float64
 13  PCT        1281 non-null   float64
 14  Diagnosis  1281 non-null   object
dtypes: float64(14), object(1)
memory usage: 150.2+ KB
```

Figure 3.5: Structure of the dataset (df.info())

Since all predictors are already numeric, no categorical encoding is needed for the input features. Only the Diagnosis column, being of object type, must be converted to integer labels before it can be used as the target variable of the classifiers. The consistent data types and the complete non-null counts mean that the dataset can be passed to the preprocessing stage without type conversion or imputation.

A duplicate-row check found 49 rows that exactly repeat an earlier row in all 15 columns. Because identical rows can end up on both sides of the train/test split, they can make test results look slightly better than they should; this is acknowledged as a limitation in Chapter 8.

```
Duplicate rows (beyond first occurrence): 49
       WBC    LYMp   NEUTp     LYMn    NEUTn   RBC   HGB      HCT    MCV  \
935   2.88  25.845  77.511  1.88076  5.14094  3.01   9.0  46.1526  103.7
951   2.88  25.845  77.511  1.88076  5.14094  3.01   9.0  46.1526  103.7
452   3.10  20.300  72.100  0.60000  2.30000  5.36  12.2  40.4000   75.5
467   3.10  20.300  72.100  0.60000  2.30000  5.36  12.2  40.4000   75.5
90    3.20  35.600  56.200  1.10000  1.80000  5.22  14.9  45.8000   87.8
441   3.20  35.600  56.200  1.10000  1.80000  5.22  14.9  45.8000   87.8
924   3.92  25.845  77.511  1.88076  5.14094  4.97  13.9  46.1526   88.1
1068  3.92  25.845  77.511  1.88076  5.14094  4.97  13.9  46.1526   88.1
451   4.70  27.800  62.300  1.30000  2.90000  4.38  12.3  38.4000   87.8
466   4.70  27.800  62.300  1.30000  2.90000  4.38  12.3  38.4000   87.8

       MCH  MCHC    PLT       PDW      PCT                      Diagnosis
935   29.9  28.8  400.0  14.312512  0.26028             Macrocytic anemia
951   29.9  28.8  400.0  14.312512  0.26028             Macrocytic anemia
452   22.7  30.1  149.0  13.600000  0.14000          Iron deficiency anemia
467   22.7  30.1  149.0  13.600000  0.14000          Iron deficiency anemia
90    28.5  32.5  132.0  14.300000  0.13000           Thrombocytopenia
441   28.5  32.5  132.0  14.300000  0.13000           Thrombocytopenia
924   28.0  31.7  229.0  14.312512  0.26028                    Healthy
1068  28.0  31.7  229.0  14.312512  0.26028                    Healthy
451   28.0  32.0  171.0  11.500000  0.15000  Normocytic normochromic anemia
466   28.0  32.0  171.0  11.500000  0.15000  Normocytic normochromic anemia
```

Figure 3.6: Duplicate records in the dataset

Table 3.4 shows the evidence. In each of these columns a single value is repeated far more often than chance would allow, and that value equals the column mean (to four decimal places).

Feature	Repeated value	Records	Share	Column mean
LYMp	25.8450	781	61.0%	25.8450
NEUTp	77.5110	781	61.0%	77.5110
LYMn	1.8808	781	61.0%	1.8808
NEUTn	5.1409	781	61.0%	5.1409
HCT	46.1526	781	61.0%	46.1526
PCT	0.2603	781	61.0%	0.2603
PDW	14.3125	417	32.6%	14.3125

Table 3.4: Repeated values consistent with mean imputation

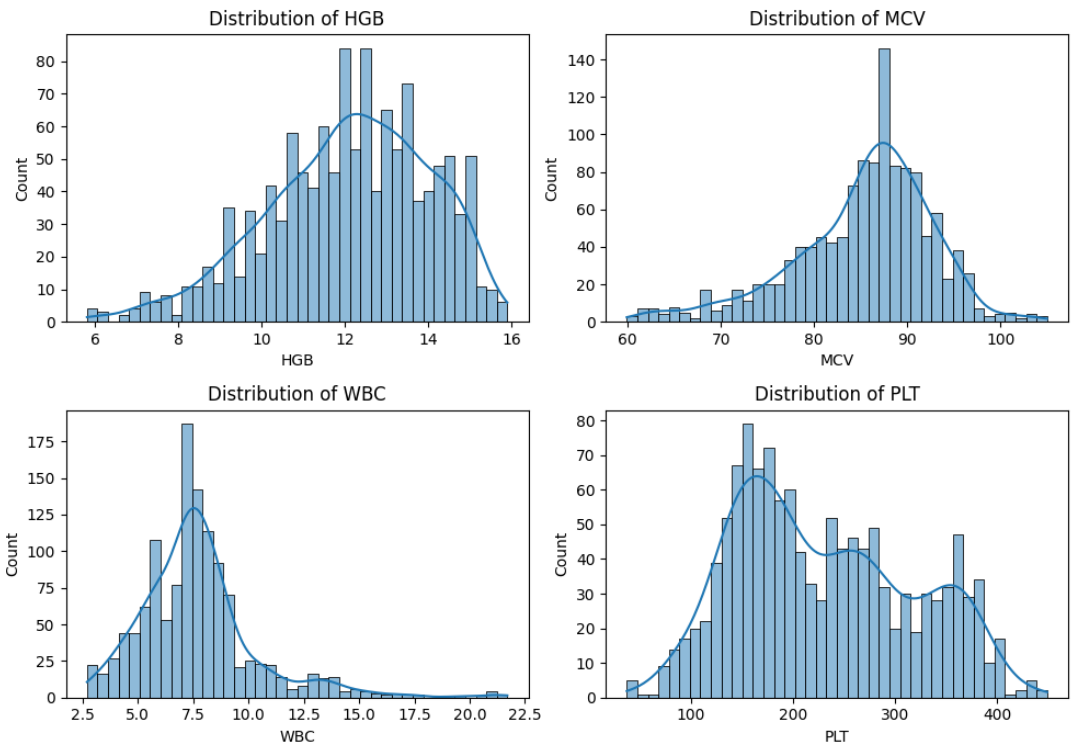


Figure 3.7: Distributions of HGB, MCV, WBC and PLT (1st to 99th percentile)

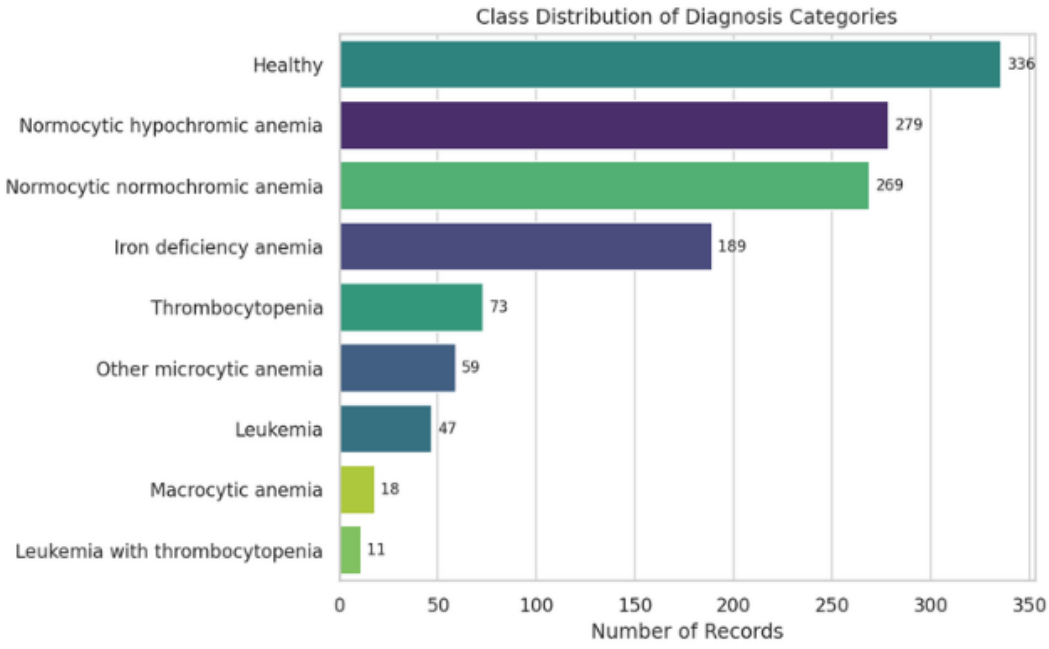


Figure 3.8: Bar chart showing the class distribution of the Diagnosis variable

The exploration confirms that all 14 features are numeric (float64), that no values are missing, and that the Diagnosis column holds nine classes. The three largest classes (Healthy, Normocytic hypochromic and Normocytic normochromic) account for 884 of the 1,281 records (about 69%),

while the two rarest (Macrocytic anemia and Leukemia with thrombocytopenia) together account for only 29 records (about 2%).

Insights from the exploratory analysis:

- The data is clean in format (all 14 features numeric, no missing values) but strongly imbalanced, with the largest class about 30 times the size of the smallest.
- Hemoglobin, red cell indices and platelet and white cell measures differ across classes in clinically expected directions, so the features carry real signal for classification.
- A small number of records contain physiologically impossible values, such as a negative hemoglobin, and 49 rows are exact duplicates.
- Several columns contain a single repeated value equal to the column mean, which indicates that missing values were filled by mean imputation before the dataset was published; this should be kept in mind when interpreting the results.

3.2 Data Preprocessing

Table 3.5 gives an overview of every preprocessing step considered in this project, what was found, and whether the step was actually applied to the data used for training and testing or only checked during analysis. The details of each step follow in the sections below.

The order of the steps is important for the validity of the results. Label encoding and the train–test split are carried out first, so that every later operation that learns from the data is fitted only on the training portion and never sees the test records. Class balancing with SMOTE is applied after the split and only to the training set, because generating synthetic samples before splitting would let information from the test set leak into training and make the reported performance unrealistically high. In the same way, the standard scaler used for Logistic Regression is fitted on the balanced training data and then applied unchanged to the test data. The test set therefore keeps the original, imbalanced class distribution, which reflects how the model would meet new patients in practice. Steps that only inspect the data, namely the checks for missing values, duplicate rows, and out-of-range values, are reported in the table together with their outcome, so that it is clear which findings led to a change in the data and which were recorded as observations about the dataset. The same dataset-level statistics were used to decide that no further transformation of the features was required for the tree-based models, since Random Forest and XGBoost are insensitive to feature scale and to monotonic transformations of the inputs.

Step	What was found	Status
Missing values	No null values in any of the 15 columns; however, seven columns contain a repeated value equal to the column mean, which indicates that the source data was mean-imputed (Table 3.4)	No imputation needed; the existing imputed values were left as they are
Duplicate rows	49 rows exactly repeat an earlier row	Checked only; not removed
Out-of-range values	38 records across 12 features fall outside generous physiological bounds, including one negative HGB value	Checked only; not removed or capped
Label encoding	Nine class names in the Diagnosis column	Applied: converted to integer labels 0 to 8 with a label encoder
Train/test split	1,281 records, imbalanced classes	Applied: stratified 80/20 split (1,024 training and 257 test samples), fixed random seed
Class balancing	Largest class about 30 times the smallest	Applied to the training set only: SMOTE with $k = 5$, giving 269 samples per class (2,421 in total)
Feature scaling	Features differ greatly in magnitude	Applied for Logistic Regression only: standard scaling fitted on the balanced training data

Table 3.5: Summary of preprocessing steps and their status

The duplicate and out-of-range records were deliberately left in the data used for the reported experiments, so that the results reflect the raw dataset as published. This is acknowledged as a limitation, and removing or capping these records is listed as future work.

3.2.1 Data Cleaning

A physiological validity check was applied to each of the 14 numeric features, using generous technical bounds that reject only impossible values (for example, a negative hemoglobin) and not abnormal but plausible ones. Table 3.6 shows the bound used for each feature and the number of values outside it. In total, 38 values across 12 features were flagged; LYMp and PLT had none. These values are consistent with data entry or unit/decimal-scale artefacts rather than true biological extremes.

Feature	Unit	Allowed range used	Values outside
WBC	$\times 10^3/\mu$L	0–30	4
LYMp	%	0–100	0
NEUTp	%	0–100	2
LYMn	$\times 10^3/\mu$L	0–20	1
NEUTn	$\times 10^3/\mu$L	0–25	2
RBC	$\times 10^6/\mu$L	0–8	5
HGB	g/dL	0–25	5
HCT	%	0–100	3
MCV	fL	0–150	2
MCH	pg	0–200	4
MCHC	g/dL	0–50	5
PLT	$\times 10^3/\mu$L	0–1000	0
PDW	fL	0–30	1
PCT	%	0–5	4
		Total	**38**

Table 3.6: Physiological bounds and the number of values outside them

The Diagnosis column holds nine class names as text, so it is converted to integer labels with a label encoder before training. The encoder assigns the numbers in alphabetical order of the class names (Table 3.7). These integer codes are used internally by the models and in the confusion matrices, and are converted back to class names in the application.

Label	Class
0	Healthy
1	Iron deficiency anemia
2	Leukemia
3	Leukemia with thrombocytopenia
4	Macrocytic anemia
5	Normocytic hypochromic anemia
6	Normocytic normochromic anemia
7	Other microcytic anemia
8	Thrombocytopenia

Table 3.7: Integer labels assigned to the nine classes

The dataset was checked for missing values across all 15 columns; no missing (null) values were found in any column, so no imputation was required. A duplicate-row check identified 49 duplicate records during exploratory analysis.

A physiological validity check was then applied to each of the 14 numeric features, using generous technical bounds for each CBC parameter (e.g., WBC: 0–30 $\times 10^3$/L, HGB: 0–25 g/dL, MCV: 0–150 fL). This flagged 38 records across 12 features as containing values outside plausible physiological range, including WBC (4 records), RBC (5), HGB (5, including one negative value), HCT (3), MCV (2), MCH (4), MCHC (5), NEUTp (2), NEUTn (2), LYMn (1), PDW (1), and PCT (4). These values are consistent with data entry or unit/decimal-scale artefacts rather than true biological extremes.

In addition to cleaning, feature normalization (standard scaling for the Logistic Regression baseline only) is applied to the feature set. This step is not required by the tree-based ensemble methods (Random Forest and XGBoost), which are invariant to monotonic feature scaling, but it is necessary for the Logistic Regression baseline, which is sensitive to differences in feature magnitude. Inspection of the raw file also shows that the absence of missing values is partly an artefact of how the dataset was prepared. In seven columns (LYMp, NEUTp, LYMn, NEUTn, HCT, PCT and, to a lesser degree, PDW), a single repeated value occurs in 781 of the 1,281 records (about 61%; 417 records or about 33% for PDW), and this value equals the column mean. This suggests that missing entries in the original source were filled with the column mean before the dataset was published. The duplicate and out-of-range records were identified but not removed before the train/test split (see Chapter 4), and this should be kept in mind when interpreting the results.

3.2.2 Analysis of Feature Variables

Each of the 14 feature variables was analysed individually for its distribution shape and clinical relevance. Table 3.8 summarises the clinical meaning of each feature and the skewness observed in its distribution prior to cleaning; a skewness value close to 0 indicates a roughly symmetric distribution, while larger positive values indicate a longer right tail, often caused by the outlier records discussed in Section 3.2.1. Clinically, HGB and the red cell indices (MCV, MCH, MCHC) are the standard indicators for classifying anemia by cell size and hemoglobin content, while the white cell and platelet measures help separate the Leukemia and Thrombocytopenia classes from the red-cell anemias. For example, the median WBC is highest in the Leukemia class (12.8) and the median platelet count is lowest in Thrombocytopenia (120) and Leukemia with thrombocytopenia (115), compared with 8.0 and 330 in the Healthy class. Because Random Forest and XGBoost split on feature thresholds, the heavy skewness of features such as NEUTp, HCT and MCV does not affect them.

Feature	Clinical Meaning	Skewness (raw)
WBC	Total white blood cell count	3.81
LYMp	Lymphocyte percentage	1.66
NEUTp	Neutrophil percentage	34.95
LYMn	Absolute lymphocyte count	22.11
NEUTn	Absolute neutrophil count	15.72
RBC	Red blood cell count	24.57
HGB	Hemoglobin concentration	11.58
HCT	Hematocrit (packed cell volume)	33.73
MCV	Mean corpuscular volume	28.60
MCH	Mean corpuscular hemoglobin	25.41
MCHC	Mean corpuscular hemoglobin concentration	8.24
PLT	Platelet count	0.45
PDW	Platelet distribution width	16.60
PCT	Plateletcrit	17.76

Table 3.8: Clinical meaning and observed skewness of each CBC feature variable

Most features show high skewness in their raw form, driven largely by the small number of outlier records identified during cleaning; PLT is the most symmetric feature, while NEUTp, HCT, and MCV are the most heavily skewed. This informs the choice of tree-based modelling (Random Forest), which does not require features to be normally distributed or scaled, unlike distance-based methods such as KNN.

3.2.3 Analysis of Class Variables

The Diagnosis variable was analysed to characterise how the feature variables differ across the nine diagnostic classes. Figure 3.9 compares the distribution of Hemoglobin (HGB) and Mean Corpuscular Volume (MCV), two of the most diagnostically relevant CBC parameters, across all nine classes.

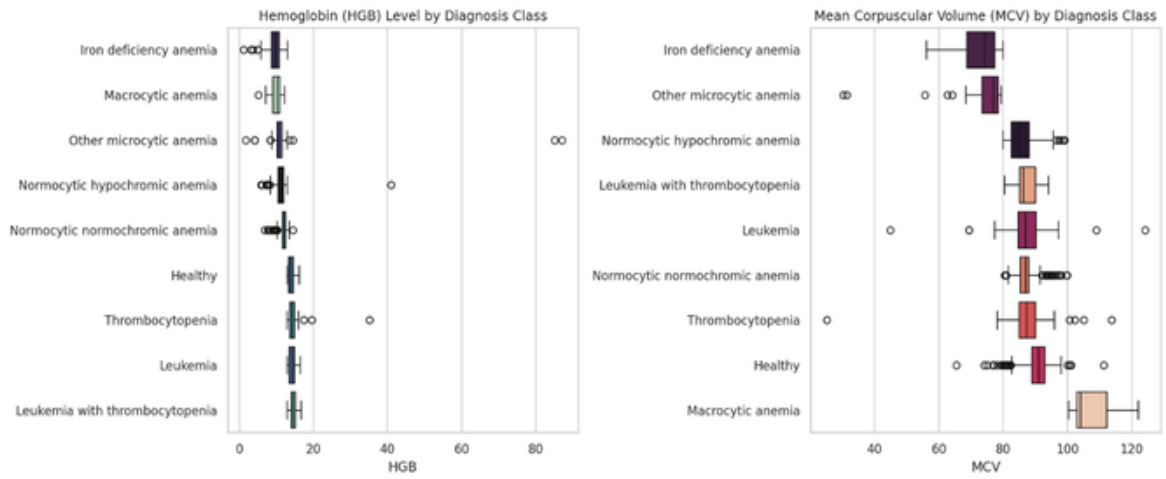


Figure 3.9: Distribution of Hemoglobin (HGB) and MCV across the nine diagnosis classes

As expected clinically, the red-cell anemia classes show clearly lower HGB than the Healthy class (median 13.9 g/dL), for example Iron deficiency anemia (9.8 g/dL), Macrocytic anemia (10.3 g/dL) and Normocytic hypochromic anemia (11.2 g/dL). The Leukemia, Leukemia with thrombocytopenia and Thrombocytopenia classes have HGB medians (14.2 to 14.6 g/dL) similar to or slightly above the Healthy class, since these conditions mainly affect white cells and platelets rather than hemoglobin. MCV distinguishes microcytic classes (such as Iron deficiency anemia and Other microcytic anemia, with lower MCV) from macrocytic classes (Macrocytic anemia, with higher MCV) and normocytic classes, confirming that the feature set carries genuine discriminative signal for the classification task.

The class imbalance noted in Section 3.1.1, with the largest class (Healthy, 336 records) outnumbering the smallest class (Leukemia with thrombocytopenia, 11 records) by roughly 30 to 1, poses a risk that a classifier trained directly on this data will be biased toward the majority classes and perform poorly on rare but clinically important subtypes such as Leukemia and Leukemia with thrombocytopenia. This is the direct motivation for applying SMOTE (Synthetic Minority Oversampling Technique) during training, as noted in the project pipeline in Section 3.5.

3.3 Data Visualization

In addition to the class-wise and feature-wise visualizations presented in Sections 3.1 and 3.2, two further visualizations were produced to support model design decisions: a correlation heatmap of the 14 feature variables, and boxplots highlighting outliers in selected features. The heatmap was used to check whether the 14 CBC parameters are redundant enough to justify dropping any of them, and the boxplots were used to visually confirm the out-of-range records flagged during the physiological validity check in Section 3.2.1 (38 records across 12 features).

Together, they support the decisions to retain all 14 features and to rely on tree-based models that do not require normally distributed or scaled inputs.

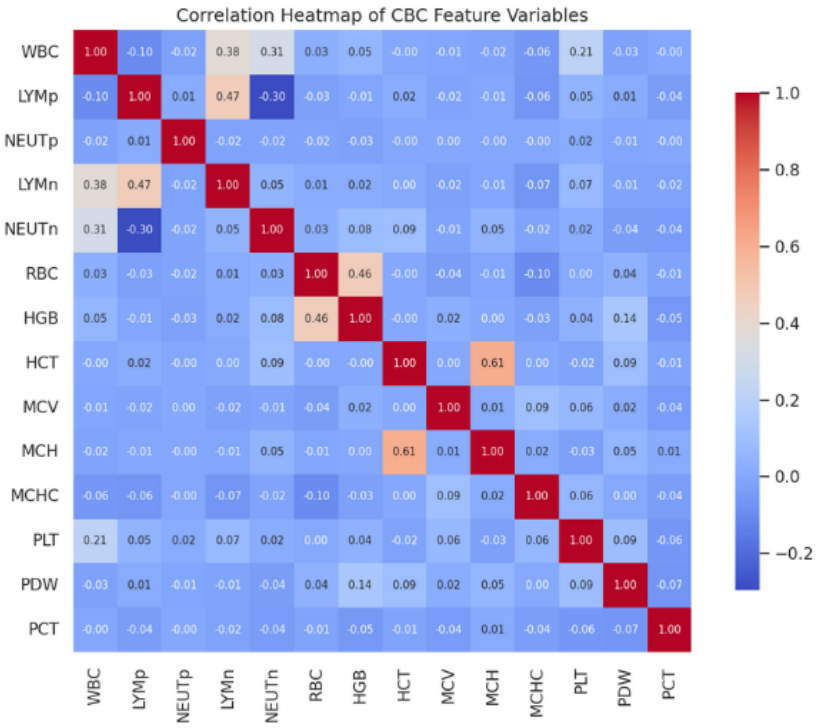


Figure 3.10: Pearson correlation heatmap of the 14 CBC features (raw data)

Figure 3.10 shows the Pearson correlation of the raw data. It suggests that the features are only weakly correlated, with the strongest pair being HCT with MCH (0.61). This result is misleading, because the raw data contain outliers and the mean-imputed constants described in Table 3.4, which weaken the coefficient. The analysis was therefore repeated with out-of-range values and imputed constants treated as missing, using Spearman rank correlation (Figure 3.11). The cleaned matrix shows four strongly correlated pairs: HGB with HCT (0.95), PLT with PCT (0.93), LYMp with NEUTp (-0.92) and WBC with NEUTn (0.88). A further pair, MCV with MCH, is also high (0.84). These relationships follow from the definitions of the parameters: hematocrit tracks hemoglobin, plateletcrit is derived from the platelet count, the two differential percentages sum to nearly 100%, and the absolute neutrophil count is a fraction of WBC. In addition, HCT is closely related to RBC and MCV ($r = 0.92$ with RBC $\times$ MCV / 10 on records with a measured HCT), which supports the consistency check used in the application.

All 14 features were nevertheless retained. Random Forest and XGBoost are not adversely affected by correlated predictors, since each split uses a random subset of features (Random Forest) or selects the best available threshold (XGBoost), and the correlated parameters are clinically meaningful in their own right. The cost is that feature importance is shared among correlated features, so individual importance values should be read as indicative (Section 5).

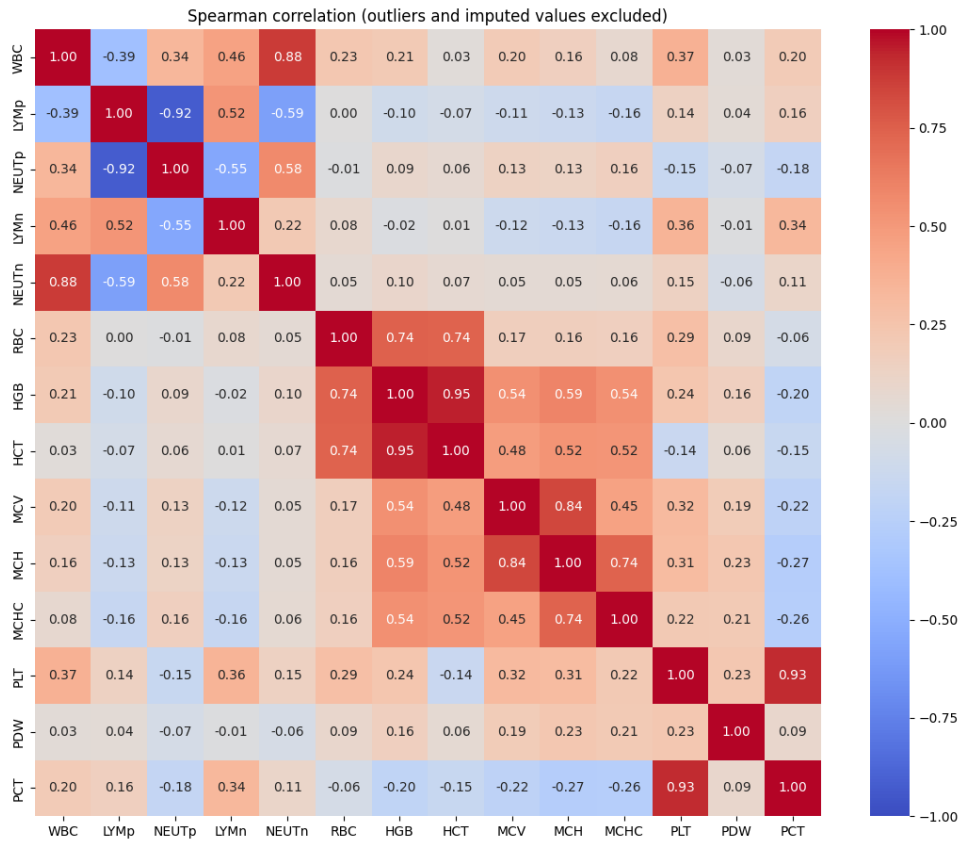


Figure 3.11: Spearman correlation heatmap of the 14 CBC features after excluding out-of-range and mean-imputed values

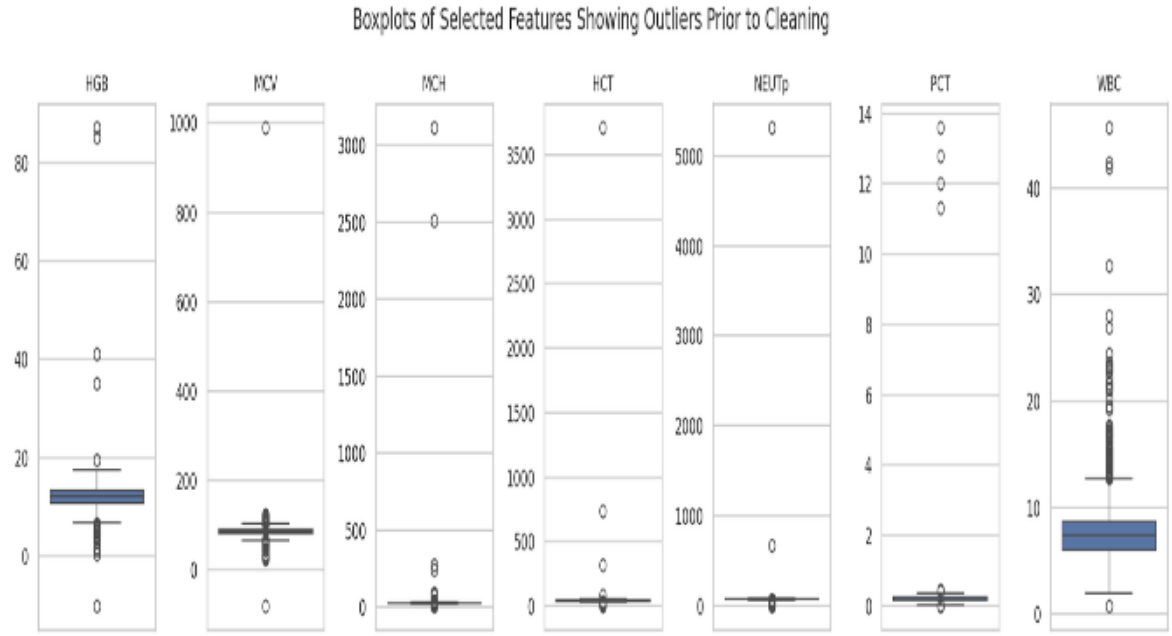


Figure 3.12: Boxplots of selected features showing outlier values prior to cleaning

The boxplots visually confirm the outlier records identified in Section 3.2.1, most clearly in HGB, MCV, MCH, HCT, NEUTp, and PCT, where isolated points lie far outside the interquartile range and, in several cases, outside physiologically valid bounds. WBC also shows a small number of high-value outliers consistent with the Leukemia and Leukemia with thrombocytopenia classes, where elevated white blood cell counts are clinically expected rather than erroneous, and are therefore retained rather than removed.

3.4 Analysis of Architecture and Algorithms

The classification component of the system is built around Random Forest as the primary classifier, selected for its robustness in handling multi-class classification problems, non-linear feature interactions, and class imbalance on tabular data. XGBoost is trained and evaluated in parallel as a comparative benchmark, to check whether a boosting-based ensemble offers meaningful uplift over the bagging-based Random Forest on this dataset. Logistic Regression is included as a third, interpretable baseline. Both Random Forest and XGBoost natively support multi-class classification with 9 output classes and require no feature scaling; only the Logistic Regression baseline depends on the normalization step described in Section 3.2.1.

All three models are trained on the SMOTE-balanced training split and evaluated on the identical held-out test dataset using accuracy, precision, recall, and weighted F1-score. The resulting performance metrics will determine the optimal classification algorithm to be serialized and implemented within the interactive Streamlit web application. Evaluation uses overall accuracy, weighted F1-score, per-class precision and recall, a 9×9 confusion matrix, and one-vs-rest ROC curves with macro-averaged AUC.

3.4.1 Detailed Study Report of the Proposed Algorithm(s)

Step-by-step working of the Random Forest algorithm:

1. Input: The cleaned and preprocessed CBC dataset, with 14 feature variables and the 9-class Diagnosis label, is split into training and testing sets.
2. Bootstrap sampling: N random samples (with replacement) are drawn from the training set, one for each tree to be grown in the forest. Each bootstrap sample is of the same size as the original training set but contains repeated and omitted records.
3. Tree growth: For each bootstrap sample, an individual decision tree is grown. At every node of the tree, instead of considering 14 features, a random subset of m features is considered for splitting.
4. Split selection: Among the randomly selected subset of features, the split that produces

the greatest reduction in Gini impurity (or equivalently, the greatest information gain) is chosen to divide the node into child nodes.

5. Recursive growth: This splitting process repeats recursively down each branch of the tree until a stopping criterion is met, such as a maximum tree depth, a minimum number of samples per leaf, or node purity.

6. Forest aggregation: Once all *N* trees have been grown, each tree independently predicts a diagnosis class for a given input record. The forest's final prediction is obtained by majority voting across all *N* trees.

7. Output: The class with the most votes is returned as the predicted diagnosis, which is one of the 9 categories (Healthy or one of the 8 anemia subtypes).

Because each tree is trained on a different bootstrap sample and considers a different random subset of features at each split, individual trees are decorrelated from one another. Aggregating their predictions by majority voting reduces the variance associated with any single decision tree, which improves generalisation and reduces the risk of overfitting, an important property given the comparatively modest size (1,281 records) and class imbalance of the dataset used in this project.

Additionally, the bagging mechanism ensures that the ensemble remains resilient to noise and extreme outlier values present in routine laboratory blood tests. By distributing the evaluation of split criteria over multiple randomized decision paths, the model prevents dominant features such as Hemoglobin or Mean Corpuscular Volume from masking subtler diagnostic signals in less prominent parameters. Furthermore, this architecture allows the calculation of Gini-based feature importance scores across the entire ensemble, providing a direct metric to rank the most discriminative hematological markers contributing to subtype differentiation. Consequently, Random Forest strikes a good balance between high predictive accuracy across all nine classes and low computational overhead, making it well suited for real-time inference.

Step-by-step working of the XGBoost algorithm:

1. Input: The preprocessed CBC dataset, containing 14 feature variables and the 9-class Diagnosis label, is partitioned into training and testing sets.

2. Initialization: The model initializes with a base prediction representing the default log-odds across all nine target classes.

3. Gradient computation: At each boosting iteration, the first-order gradients and second-order hessians of the multi-class cross-entropy loss function are calculated relative to the existing ensemble's predictions.

4. Tree fitting: A new base regression tree is constructed and fit directly to the pseudo-residuals (negative gradients) to correct the errors of the current ensemble.

5. Regularization: To prevent overfitting, shrinkage (learning rate) is applied to scale the contribution of each new tree, alongside L1 and L2 penalty terms on leaf weights and tree depth complexity.
6. Ensemble update: The ensemble prediction is updated by adding the product of the learning rate and the newly fitted tree's output to the previous cumulative score.
7. Stopping check: Iterative boosting cycles repeat sequentially until the specified maximum number of estimators is reached or convergence criteria are satisfied.
8. Output: Final multi-class probability distributions are computed via a softmax function across the accumulated scores, assigning the instance to the highest-probability category among the nine classes.

Unlike Random Forest, which constructs trees independently and in parallel via bagging to reduce variance through averaging, XGBoost generates trees sequentially via gradient boosting, with each subsequent tree focusing on the residual misclassifications of the preceding model. While XGBoost is more sensitive to hyperparameter tuning it often achieves fine boundary resolution on complex tabular datasets such as the complete blood count panel analyzed here. Evaluating it alongside Random Forest provides a direct comparative benchmark between bagging and boosting ensemble paradigms.

Logistic Regression is included as an interpretable linear baseline rather than a primary candidate. It models the log-odds of each diagnostic class as a linear combination of the scaled input features using a multinomial softmax function. This model serves two distinct functions in the project: first, it provides a fast-to-train sanity check to verify that non-linear ensemble architectures yield significant performance uplift over a linear decision boundary; and second, it offers a straightforward reference framework where the magnitude and sign of learned feature coefficients illustrate the relative directional influence of individual blood parameters across the diagnostic categories.

Step-by-step working of the Logistic Regression algorithm:

- Input: The cleaned, preprocessed CBC dataset, with 14 feature variables and the 9-class Diagnosis label, is partitioned into training and testing sets, with features standardized (zero mean, unit variance) prior to training.
- Initialization: Model weights (coefficients) for each of the 9 classes are initialized, typically to zero or small random values, along with a bias term per class.
- Linear combination: For each input record, a linear combination of the scaled feature values is computed for every class, producing a raw score (logit) per diagnostic category.
- Softmax transformation: The logits across all 9 classes are passed through a multinomial softmax function, converting them into a normalized probability distribution that sums to

1 across all diagnostic categories.

- Loss computation: The multinomial cross-entropy loss is calculated between the predicted class probabilities and the true one-hot encoded diagnosis label.
- Gradient descent update: Gradients of the loss with respect to each class's weights and bias are computed, and the parameters are updated iteratively in the direction that minimizes the loss, scaled by the learning rate.
- Convergence check: Steps 3–6 repeat across multiple iterations (epochs) until the loss stabilizes below a convergence threshold or the maximum number of iterations is reached.
- Output: The final class probabilities are computed via softmax over the learned linear scores, and the diagnosis with the highest probability is returned as the predicted class among the 9 categories.

Unlike Random Forest and XGBoost, which model non-linear decision boundaries through recursive partitioning of the feature space, Logistic Regression assumes a linear relationship between the input features and the log-odds of each class. This makes it substantially faster to train and directly interpretable — the sign and magnitude of each coefficient indicate whether a feature increases or decreases the likelihood of a given diagnosis — but it is less capable of capturing the complex, non-linear interactions between CBC parameters that the ensemble methods can exploit, which is reflected in its comparatively lower accuracy in this project's results.

3.5 Project Pipeline

Figure 3.12 illustrates the end-to-end pipeline of the proposed system, from raw data acquisition through to deployment as a usable web application.

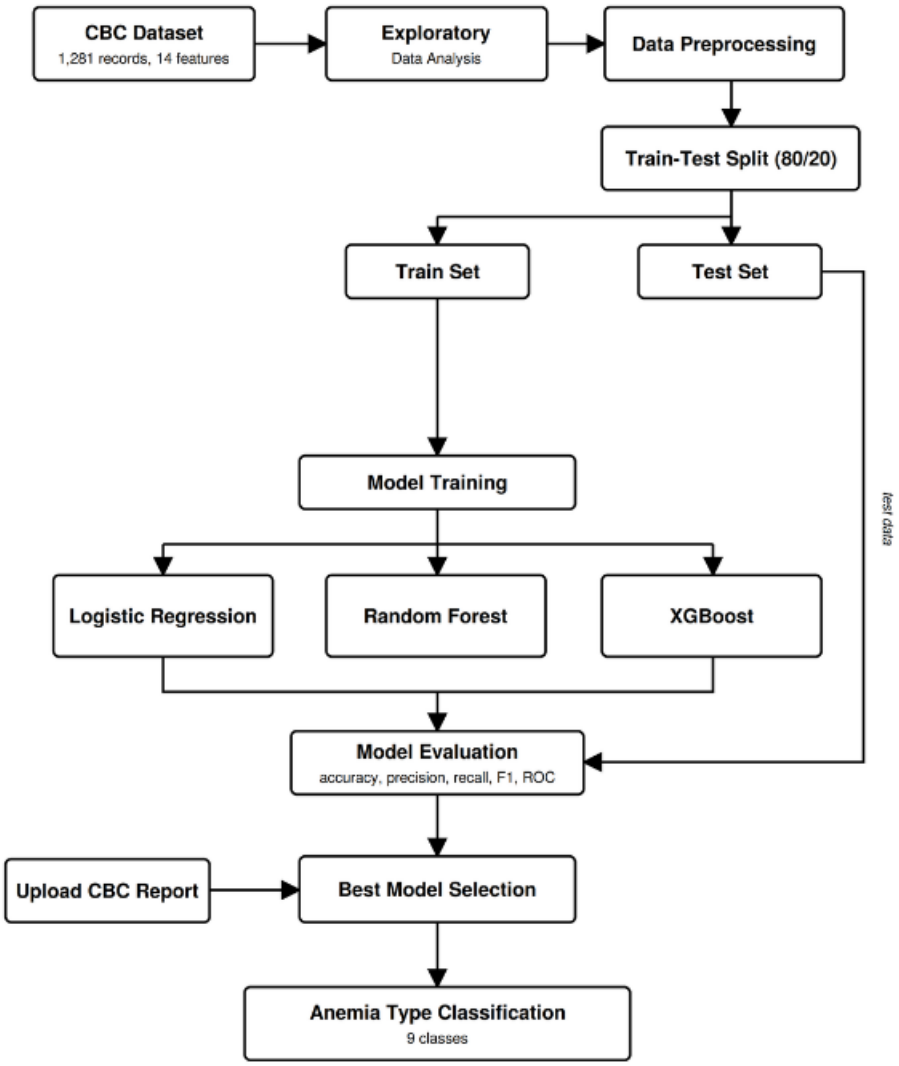


Figure 3.13: End-to-end project pipeline

- CBC Dataset (Data Collection): The raw Complete Blood Count (CBC) dataset forms the initial entry point of the pipeline. It comprises 1,281 clinical records across 14 hematological parameters, serving as the foundational input for learning discriminative patterns across different anemia subtypes.
- Exploratory Data Analysis (EDA): In this stage, statistical distributions, parametric skewness, and inter-feature correlations are examined to assess data behavior and biological relationships. It also identifies the natural class distribution and highlights severe frequency disparities across rare blood disorders.
- Data Preprocessing: Features are standardised for Logistic Regression; duplicate and out-of-range checks were performed for analysis but not applied.
- Train-Test Split (80/20, Stratified): The preprocessed dataset is partitioned into an 80% training set and a 20% independent testing set using stratified sampling. Stratification is

essential to ensure that every diagnostic subtype, especially sparsely represented minority categories, maintains identical proportional representation in both splits.

- Train Set & Test Set Isolation: The data is strictly isolated into two distinct pathways to prevent data leakage. The test set is completely held out and reserved exclusively for unbiased final evaluation, while all balancing and fitting steps are restricted to the training pathway.
- SMOTE Oversampling (k=5, train set only): Synthetic Minority Over-sampling Technique (SMOTE) with 5 nearest neighbors is applied solely to the training partition. It creates synthetic feature vectors for underrepresented anemia classes, neutralizing the severe $\sim$30:1 class imbalance without discarding majority data or causing leakage into the test set.
- Model Training: Three diverse classification architectures are trained in parallel on the balanced training split:
  - Logistic Regression (StandardScaler): Employs standard scaling to normalize feature magnitudes and acts as an interpretable linear baseline.
  - Random Forest (Primary Model): A bagging ensemble of decorrelated decision trees that evaluates random feature subspaces to handle non-linear boundaries and feature collinearity natively.
  - XGBoost (Ensemble Benchmark): A regularized sequential gradient boosting framework that minimizes residual errors to serve as an ensemble performance benchmark.
- Model Evaluation (Accuracy, F1-score): The trained models are evaluated on the unseen, un-sampled 20% test set. Performance is comprehensively assessed using classification accuracy, precision, recall, and weighted/macro F1-scores, alongside a 9×9 confusion matrix to verify minority-class sensitivity.
- Best Model Selection (Random Forest): Based on the multi-class evaluation metrics, the best-performing architecture is identified. Random Forest is selected as the primary deployment model due to its superior overall accuracy (98.83%), balanced F1-score (98.89%), and resistance to tabular overfitting.
- Deployment (Upload CSV/Excel or Enter Values): The selected model is integrated into a lightweight web interface. Laboratory technicians or clinicians can upload a CSV or Excel file with the 14 standard CBC parameters, or enter the values manually.
- RF Model Inference: The input parameters are verified, formatted, and passed to the serialized Random Forest model engine to execute real-time inference.
- Anemia Type Classification (9 Diagnostic Classes): The pipeline concludes by outputting the specific diagnostic prediction. The patient's blood profile is classified into 1 of the 9

validated categories: a healthy control state or one of eight specific anemia subtypes (such as Iron Deficiency, Macrocytic, or Leukemia-associated anemia).

3.6 Feasibility Analysis

Before committing to the design and implementation of the system, it is necessary to establish whether the proposed solution can realistically be built, used, and sustained with the resources available. A feasibility analysis answers this question by examining the project from several practical viewpoints instead of relying only on the accuracy of the model. This is especially important for a medical screening application, where a technically correct model is of little value if the required data are unavailable, if the intended users cannot operate the tool, or if the cost of developing and running it is out of proportion to the benefit it offers.

The problem addressed by this project is the classification of a patient's blood profile into one of nine diagnostic classes using only the routine Complete Blood Count. The CBC is one of the most frequently requested and least expensive laboratory tests, and its results are produced automatically by hematology analyzers. Interpreting these values to distinguish the types of anemia and related blood disorders, however, depends on the experience of the person reading the report, and the patterns can overlap, particularly between the microcytic and normocytic classes. A machine learning tool that suggests the likely class from values already available could therefore support earlier and more consistent screening, which is the motivation for assessing its viability here.

The analysis is based on what was actually built in this project, namely the Kaggle dataset, the open-source software stack, the three trained models, and the Streamlit application, and not on assumptions about a future production system. Three aspects are considered in turn: technical feasibility, which asks whether the models can be developed and run with the available tools; operational feasibility, which asks whether the intended users can work with the system in a realistic setting; economic feasibility, which compares the cost of development and operation with the expected benefit; Each subsection below gives its findings, and together they indicate whether the project is worth pursuing.

3.6.1 Technical Feasibility

The dataset (1,281 records, 14 features) is of modest size, so development and training are feasible with standard CPU-based computing and do not depend on pretrained deep learning models or GPU acceleration. All three models (Random Forest, XGBoost and Logistic Regression) were trained in Google Colaboratory using open-source Python libraries, namely Pandas, NumPy, Scikit-learn, XGBoost and imbalanced-learn, and the full pipeline from preprocessing to evaluation ran on CPU alone. The trained Random Forest, along with the information the

application needs, is stored as a single `joblib` file, so the Streamlit application loads it directly without retraining at start-up and produces predictions in real time. The minimum requirements are modest (a dual-core 2 GHz processor, 4 GB of RAM and 1 GB of free disk space, as listed in Chapter 10), and no specialised hardware is needed. This confirms that the project is technically achievable within the available timeframe and resources.

The technical feasibility is also supported by the results obtained. The Random Forest model reached an accuracy of 98.83% on the held-out test set, which shows that the 14 routine CBC parameters contain enough information to separate the nine classes reliably, even for a model with default hyperparameters. Training the models requires only a few minutes on CPU, so the pipeline can be rerun whenever the dataset is changed or extended. The chosen algorithms are also well established: Random Forest and XGBoost are widely used for structured tabular data, and Logistic Regression provides a simple and interpretable baseline. The use of mature, well-documented libraries lowers the technical risk of the project, because the implementation relies on tested components and does not require custom numerical code. The main technical limitations are the small number of samples in the rarest classes and the fact that the data come from a single public source, and these are discussed later as limitations of the work.

3.6.2 Operational Feasibility

The proposed web interface is designed to be simple enough for a laboratory technician or clinician to use without specialised machine learning knowledge. CBC values can be entered manually in a form with one field for each of the 14 parameters, or uploaded as a CSV or Excel file for batch prediction, and the system returns the predicted diagnosis together with the top class probabilities and a plain-language explanation. Built-in safeguards support everyday use: input fields reject values outside physiological limits, a warning appears when the entered hematocrit is inconsistent with RBC and MCV, and a "close call" warning is shown when the two most probable classes are nearly tied. Because CBC testing is already routine clinical practice, integrating this tool into existing workflows as a preliminary screening aid does not require new data collection infrastructure. The tool is intended to support, not replace, clinical judgment.

The system is also operationally practical because its input matches information that laboratories already produce. Automated hematology analyzers output all 14 parameters in a single run, so a user can copy the values directly from the report without any additional test or measurement. The batch upload option allows many patient records to be screened at once, which suits a laboratory that processes samples in groups. Prediction takes only a fraction of a second, so it does not slow down the reporting process. The reference ranges and the descriptions of the anemia types are available inside the application, so that a user can check a result without consulting another source. Training for new users is minimal because the interface follows a simple enter, predict,

and read-the-result sequence. The main operational condition is that the output is treated as a screening suggestion that must be confirmed by a clinician through further tests.

3.6.3 Economic Feasibility

All software components used in this project, including Python, Scikit-learn, XGBoost, imbalanced-learn and Streamlit, are open-source and free to use, and the dataset is publicly available on Kaggle at no cost. Training was carried out on Google Colaboratory and the source code is hosted on GitHub, so no paid software licences or dedicated servers were needed. The system does not require paid cloud infrastructure for the scale of this project and can be run on modest hardware, making the solution economically feasible for an academic mini-project and a plausible low-cost aid in resource-constrained clinical settings.

The only recurring cost of the project is the time needed for development and maintenance, since no component of the stack requires a subscription or licence fee. Because CBC is one of the most commonly performed and least expensive laboratory tests, the tool adds no extra testing cost for the patient. By suggesting the likely anemia type from a test that has already been done, it may help a clinician decide earlier which confirmatory tests, such as an iron profile, vitamin B12 and folate levels, or a peripheral smear, are worth ordering. The development effort is therefore justified by the low cost of building and running the system.

3.7 System Environment

The development of the proposed system relied on cloud-based computing resources for model development, together with a local code editor and Git for the web application and version control. This environment supports the full machine learning pipeline, from data preprocessing through model evaluation and the Streamlit application. The minimum software and hardware requirements are listed in Chapter 10.

The environment was organised around a clear separation between the development stage and the usage stage. In the development stage, the data were explored, the models were trained and compared, and the best model was exported, all inside a cloud notebook. In the usage stage, only the exported model file and a small set of libraries are needed, so the application does not depend on the training notebook or on the original dataset being present. This separation keeps the deployed component light, makes the results easier to reproduce, and allows the model to be retrained and replaced without changing the interface code. The choice of widely used open-source tools also means that every component is well documented and can be installed on a new machine with standard package management commands.

3.7.1 Hardware Environment

The diagnostic pipeline was developed and validated entirely on standard central processing unit (CPU) architecture, with no dedicated graphical processing unit (GPU) required. Because the system processes structured tabular complete blood count (CBC) data rather than high-dimensional image or unstructured text data, models such as Random Forest, XGBoost and Logistic Regression train and run efficiently on CPU threads. Storage demands are minimal, as the dataset (1,281 records) and the serialized model file are small. The Streamlit application runs on modest hardware, with a dual-core 2 GHz processor, 4 GB of RAM and 1 GB of free disk space as the minimum.

Two separate hardware contexts should be distinguished. The first is the cloud runtime used for model development, in which Google Colaboratory provides a managed virtual machine with CPU, memory, and temporary storage, so no local installation of the machine learning libraries was needed for training and evaluation. Since the notebook session is temporary, the dataset is kept in cloud storage and read from there at the start of each session, and the trained model file is exported before the session ends. The second context is the machine on which the application runs. Prediction with a trained Random Forest on a single record, or on a batch of a few hundred records uploaded from a file, takes a fraction of a second on a single CPU core, and the model file loads into memory in a short time. The memory requirement is dominated by the Python interpreter and the loaded libraries, not by the model or the data, which explains why the minimum requirements are low. A standard internet connection is needed only to access the cloud notebook during development; once the application and its libraries are installed, it can be used locally without a network connection because all processing is done on the local machine and no patient data are sent to an external server.

3.7.2 Software Environment

The software environment is based on Python (version 3.9 or later), with model development executed in Google Colaboratory. Core data manipulation, statistical profiling and cleaning are handled using Pandas and NumPy, while exploratory data distributions and model evaluation curves are rendered with Seaborn and Matplotlib. Model development and metrics computation (including weighted and macro-averaged F1-scores and confusion matrices) are carried out with Scikit-learn, augmented by XGBoost for gradient-boosted decision trees. Class balancing is handled strictly within the training partition using the imbalanced-learn library implementing SMOTE. A fixed random seed (`random_state=42`) is used for the data split, SMOTE and all three models, so the results can be reproduced. The trained Random Forest is saved with Joblib and served through a Streamlit web application, which can be opened in any modern web browser. The application code was written in VS Code or any code editor, and the project is versioned with Git and hosted on GitHub, with the libraries needed to run the application listed

in the `requirements.txt` file in the repository.

The role of each library follows the stages of the pipeline. Pandas reads the CSV file into a data frame and supports the duplicate and missing-value checks, NumPy provides the numerical operations, and Seaborn and Matplotlib produce the class distribution plots, correlation heatmap, confusion matrices, and ROC curves, which were saved as high-resolution images for use in the report and presentation. Scikit-learn supplies the stratified train–test split, the label encoder, the standard scaler used for Logistic Regression, the Random Forest and Logistic Regression classifiers, and the evaluation metrics, so that all three models are compared under identical conditions. XGBoost provides the gradient-boosted tree model as a stronger ensemble benchmark, while imbalanced-learn provides SMOTE, which is applied after the split and only to the training data to prevent leakage. Joblib serialises the trained model into a single file that Streamlit loads when the application starts. Streamlit was chosen because it allows an interactive interface, with input forms, file upload, tables, and charts, to be written entirely in Python, without separate front-end or back-end code, which keeps the application small and easy to maintain. Because the application is served locally through the browser, no web server configuration is required. Listing the library versions in `requirements.txt` ensures that the same environment can be recreated by running a single installation command, and keeping the code under Git version control records every change to the notebook and the application, which makes it possible to return to an earlier working version if a modification causes a problem.

4 System Design

4.1 Models Building

Classical machine learning was chosen over deep learning because the dataset is small (1,281 records) and tabular, a setting where tree-based ensembles and linear models typically perform as well as or better than deep neural networks, with lower risk of overfitting and more interpretable results. Three models were built on the same data split so that their results are directly comparable: Random Forest as the primary model, XGBoost as an ensemble benchmark, and Logistic Regression as an interpretable linear baseline. A fixed random seed (`random_state=42`) is used for the split, SMOTE and all three models, so the experiment can be reproduced.

4.1.1 Implementation Code

The dataset (`diagnosed_cbc_data_v4.csv`) contains 1,281 records, 14 numerical CBC features (WBC, LYMp, NEUTp, LYMn, NEUTn, RBC, HGB, HCT, MCV, MCH, MCHC, PLT, PDW, PCT) and one categorical target, Diagnosis, with 9 classes. The complete modeling code is given below.

```
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.linear_model import LogisticRegression

X = df.drop('Diagnosis', axis=1) # 14 CBC features
le = LabelEncoder()
y_encoded = le.fit_transform(df['Diagnosis']) # 9 classes -> labels 0..8

X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded)

smote = SMOTE(random_state=42, k_neighbors=5)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_smote)
X_test_scaled = scaler.transform(X_test)
```

```
rf_model = RandomForestClassifier(random_state=42)
xgb_model = XGBClassifier(random_state=42, eval_metric='mlogloss')
lr_model = LogisticRegression(max_iter=1000, random_state=42)
```

The code follows the order of the pipeline. The 14 CBC columns are separated from the Diagnosis target, and the nine class names are converted to integer labels. The data is then split 80/20 with stratification, SMOTE is fitted on the training portion only, and a StandardScaler is fitted on the SMOTE-balanced training data and applied to the test data, to be used by Logistic Regression alone. Finally, the three models are created with their settings: Random Forest and XGBoost with default hyperparameters (`random_state=42`, plus `eval_metric='mlogloss'` for XGBoost), and Logistic Regression with `max_iter=1000`.

4.1.2 Model Planning (Training/Testing Split of the Dataset)

Label encoding: The class names were converted to integers: Healthy = 0, Iron deficiency anemia = 1, Leukemia = 2, Leukemia with thrombocytopenia = 3, Macrocytic anemia = 4, Normocytic hypochromic anemia = 5, Normocytic normochromic anemia = 6, Other microcytic anemia = 7, Thrombocytopenia = 8.

Feature selection: All 14 features were retained. Several feature pairs are strongly correlated after excluding out-of-range and mean-imputed values (HGB–HCT, PLT–PCT, LYMp–NEUTp and WBC–NEUTn, all above 0.85 in Spearman correlation; Section 3.3), but they were kept, because tree-based models are robust to correlated features and the parameters are clinically meaningful.

Dataset overview: The dataset has 1,281 records and no missing values. It is imbalanced: the largest class (Healthy, 336 records) is 30.55 times larger than the smallest (Leukemia with thrombocytopenia, 11 records).

Stratified train/test split: The data was divided 80% for training and 20% for testing (`test_size=0.2`, `random_state=42`) with stratification on the diagnosis label. Stratification keeps each class in the same proportion in both subsets, which matters for the rare classes. The split gave 1,024 training samples and 257 test samples.

Class balancing with SMOTE: The Synthetic Minority Over-sampling Technique (SMOTE) creates new minority-class samples by interpolating between an existing sample and one of its $k = 5$ nearest neighbours in the same class. It was applied to the training set only. Resampling the test set would make the results look better than they really are, since synthetic samples derived from the same data would be evaluated. After SMOTE every class has 269 training samples, so the training set grows from 1,024 to 2,421 samples. The value of 269 is the size of the largest class in the training set (Healthy), so SMOTE raises every minority class to that size without removing any real records. Because stratification and SMOTE leave the test set

unchanged, it keeps the original class imbalance: the rarest classes contribute very few test samples (4 for Macrocytic anemia and 2 for Leukemia with thrombocytopenia), so performance on those classes should be read with caution.

Class	Total	Train	SMOTE	Test
Healthy	336	269	269	67
Normocytic Hypo.	279	223	269	56
Normocytic Normo.	269	215	269	54
Iron Deficiency	189	151	269	38
Thrombocytopenia	73	58	269	15
Other Microcytic	59	47	269	12
Leukemia	47	38	269	9
Macrocytic	18	14	269	4
Leukemia+Thromb.	11	9	269	2
Total	1,281	1,024	2,421	257

Table 4.1: Class distribution across the dataset splits

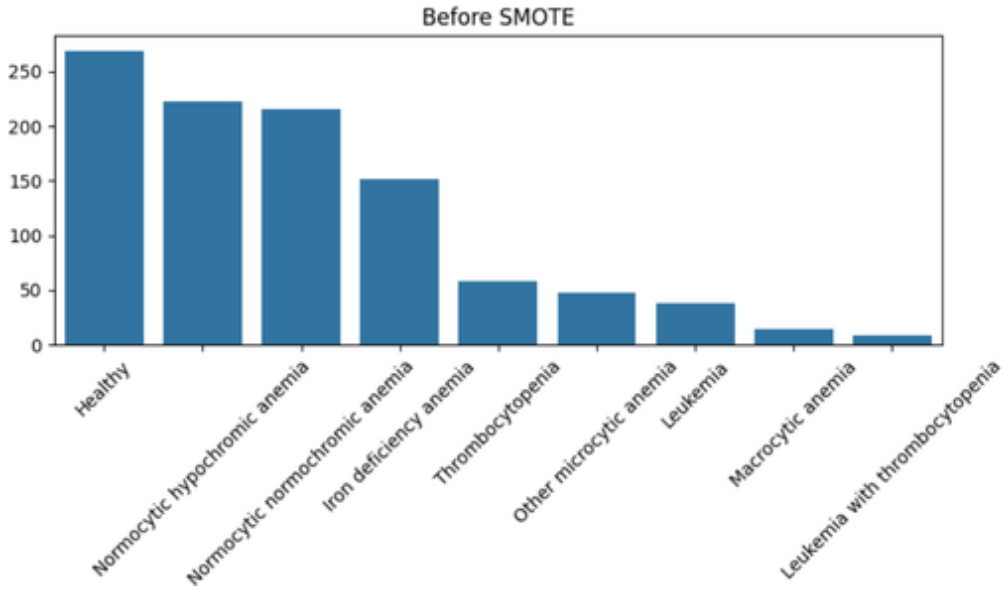


Figure 4.1: Class distribution before SMOTE

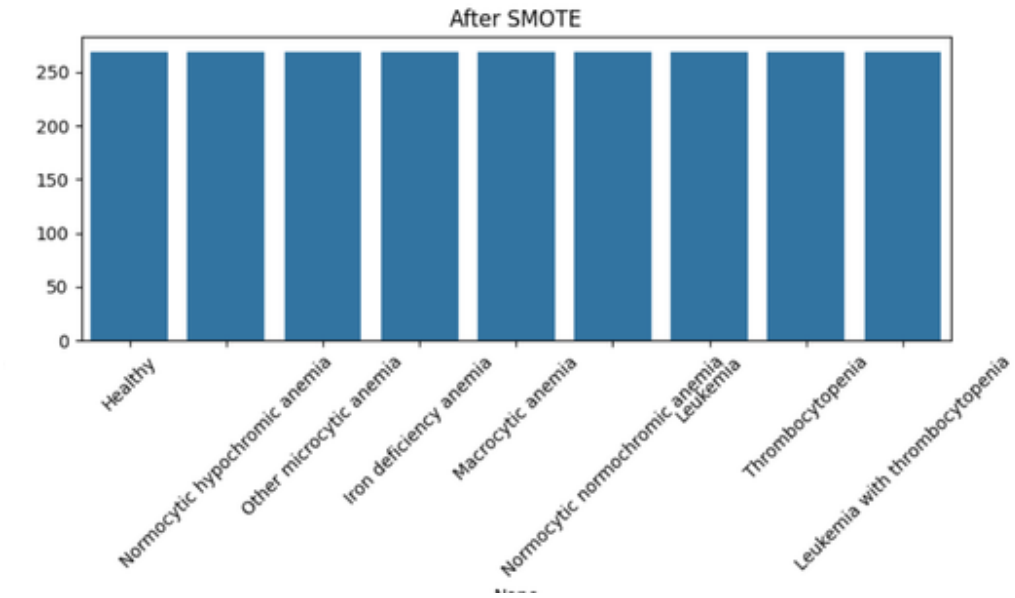


Figure 4.2: Class distribution after SMOTE

Feature scaling: A StandardScaler (zero mean, unit variance) was fitted on the training data and applied to both subsets, but only for Logistic Regression, whose optimisation is sensitive to feature magnitudes. Random Forest and XGBoost split on thresholds and are unaffected by scale, so they use the unscaled data.

Data quality note: Duplicate detection (49 duplicate rows) and physiological-bound checking (38 flagged values across 12 features) were carried out as part of the data-quality analysis. They were not applied to the split above, which uses the raw feature matrix. This should be acknowledged in the limitations.

4.1.3 Training

The models were trained on the SMOTE-balanced training set.

```
rf_model.fit(X_train_smote, y_train_smote) # Random Forest, unscaled
xgb_model.fit(X_train_smote, y_train_smote) # XGBoost, unscaled
lr_model.fit(X_train_scaled, y_train_smote) # Logistic Regression, scaled
```

Random Forest (primary model):

- An ensemble of 100 decision trees (Scikit-learn default `n_estimators=100`).
- Each tree is trained on a bootstrap sample of the data and considers a random subset of features at each split. Final predictions are made by majority vote.
- It handles non-linear relationships, is robust to outliers and correlated features, and reduces overfitting compared with a single tree.

XGBoost (ensemble benchmark):

- A gradient-boosting model with 100 trees (`n_estimators=100`), learning rate 0.3, `subsample=1.0` and `colsample_bytree=1.0` (defaults).
- Trees are added sequentially, each correcting the errors of the previous ones. The multi-class log-loss (`mlogloss`) is the evaluation metric.

Logistic Regression (baseline):

- A linear multinomial classifier with `max_iter=1000`, trained on the standardised features.
- It provides an interpretable reference for judging how much the non-linear ensembles improve on a linear decision boundary.

4.1.4 Testing

The trained models were evaluated on the 257-sample test set, which was not used for training, SMOTE or scaler fitting (except that the fitted scaler was applied to it).

```
rf_preds = rf_model.predict(X_test)
xgb_preds = xgb_model.predict(X_test)
lr_preds = lr_model.predict(X_test_scaled)
```

5 Results and Discussion

Comparison of Models

Overall performance on the test set

Model	Accuracy	Weighted F1	Correctly classified	Misclassified
Random Forest	98.83%	98.89%	254	3
XGBoost	96.89%	96.62%	249	8
Logistic Regression	74.32%	75.21%	191	66

Table 5.1: Overall performance of the three models on the test set

- Random Forest performed best, with 98.83% accuracy and a weighted F1 of 98.89%. Only 3 of the 257 test samples were misclassified.
- XGBoost was close behind at 96.89% accuracy. However, it failed completely on Leukemia with thrombocytopenia (F1 = 0.00). This class has only 2 test samples, so the result reflects just two cases and should be interpreted with caution.
- Logistic Regression reached only 74.32% accuracy, about 24.5 percentage points below Random Forest. This gap suggests that the boundaries between diagnostic classes in CBC space are non-linear and cannot be captured well by a linear model.

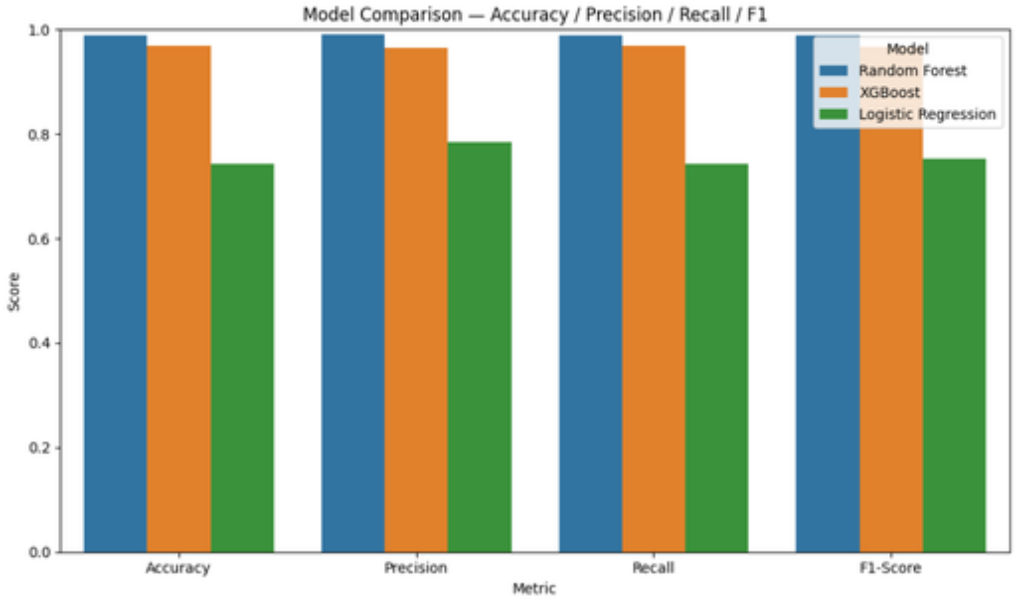


Figure 5.1: Model comparison on accuracy, precision, recall and F1-score

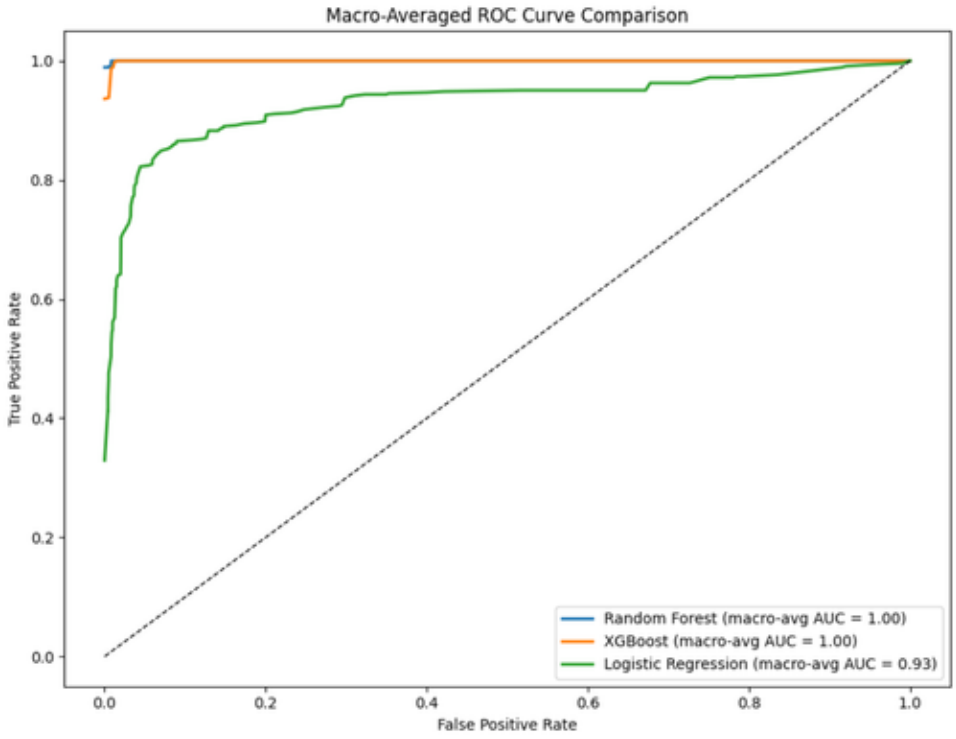


Figure 5.2: Macro-averaged ROC curve comparison of the three models

Evaluation Metrics

The following metrics were used. For a given class, TP, FP, TN and FN denote true positives, false positives, true negatives and false negatives.

Metric	Formula	Meaning
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Overall fraction of correct predictions
Precision	$\frac{TP}{TP+FP}$	Of the samples predicted as a class, how many truly belong to it
Recall (Sensitivity)	$\frac{TP}{TP+FN}$	Of the samples truly in a class, how many were found
Specificity	$\frac{TN}{TN+FP}$	Of the samples not in a class, how many were correctly rejected
F1-score	$\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$	Harmonic mean of precision and recall

Table 5.2: Evaluation metrics used in this study

- Macro average: the unweighted mean across the 9 classes. Every class counts equally, so it reveals weak performance on rare classes.
- Weighted average: the mean weighted by class support. It reflects overall performance on the actual class distribution.
- ROC curve and AUC: computed one-vs-rest for each class and macro-averaged. The AUC measures the model's ability to rank a sample of a class above samples of other classes, independent of a decision threshold.

In a medical setting recall is especially important, because a false negative means a diseased patient is classified as healthy or as the wrong condition. Accuracy alone can hide poor performance on rare classes.

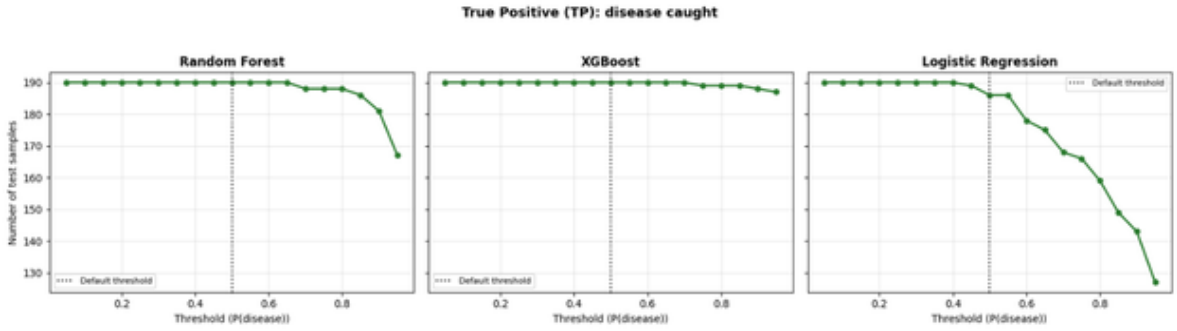


Figure 5.3: True positives (disease caught) versus decision threshold

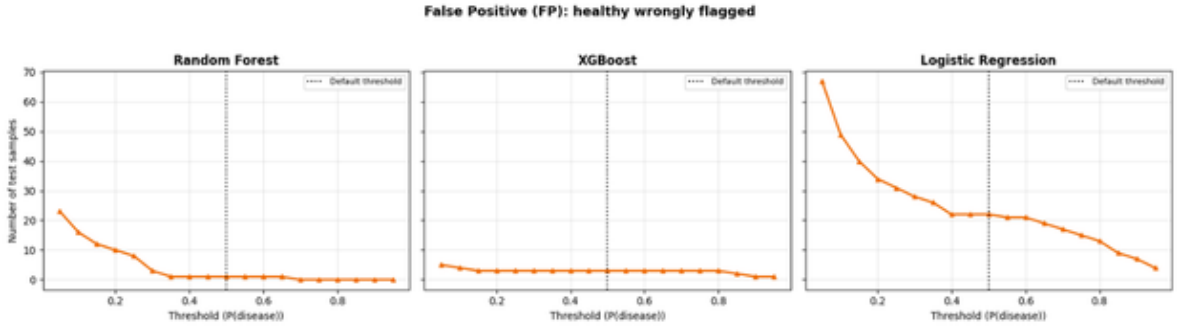


Figure 5.4: False positives (healthy wrongly flagged) versus decision threshold

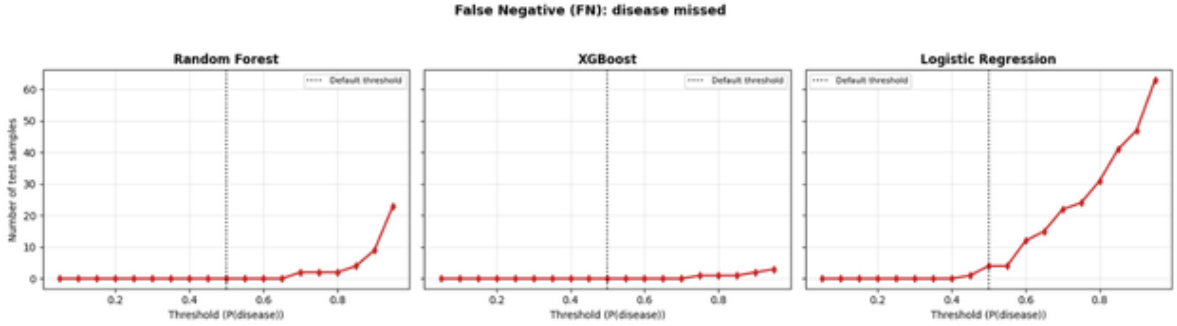


Figure 5.5: False negatives (disease missed) versus decision threshold

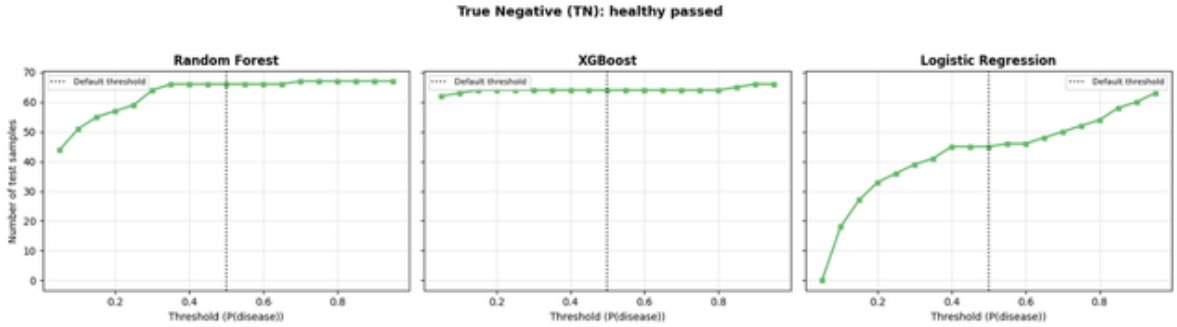


Figure 5.6: True negatives (healthy passed) versus decision threshold

Confusion Matrix

A confusion matrix for a 9-class problem is a 9×9 table. Rows are the actual diagnoses and columns are the predicted diagnoses, so the diagonal holds the correct predictions and off-diagonal cells show which classes are confused with each other.

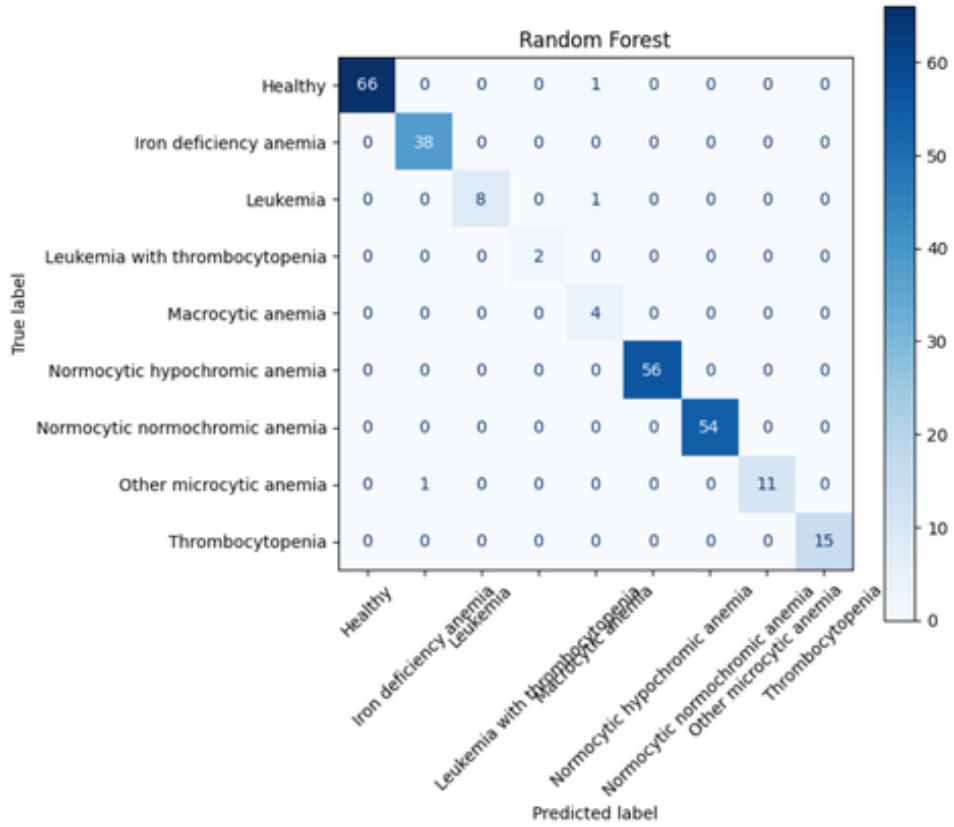


Figure 5.7: Confusion matrix of the Random Forest model

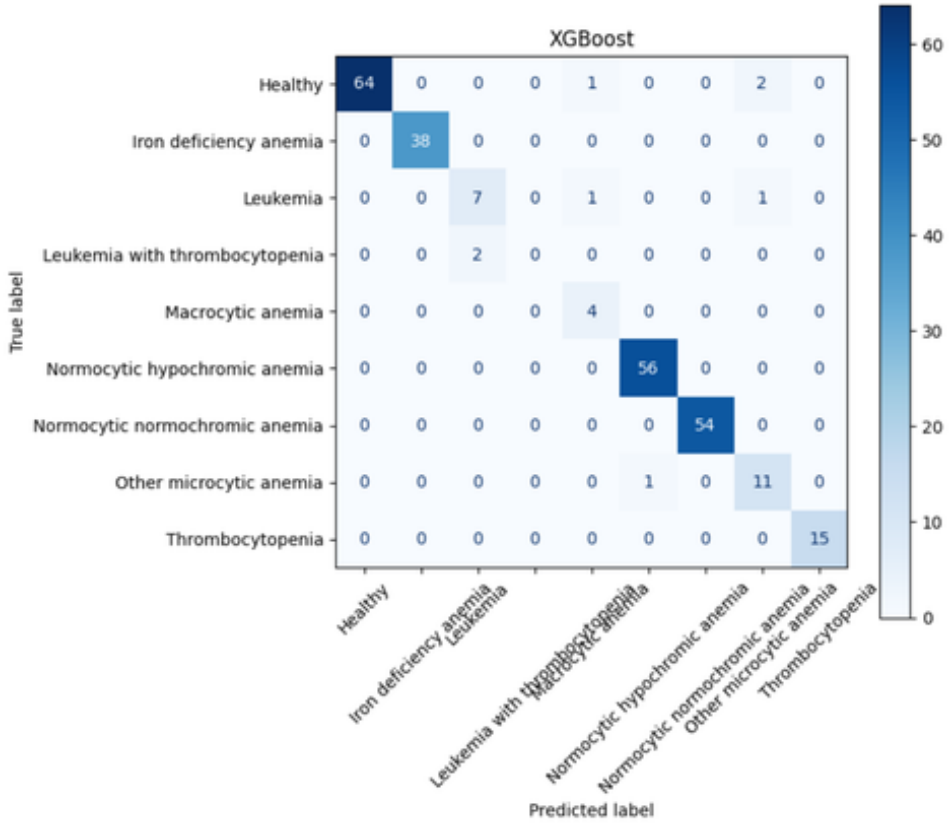


Figure 5.8: Confusion matrix of the XGBoost model

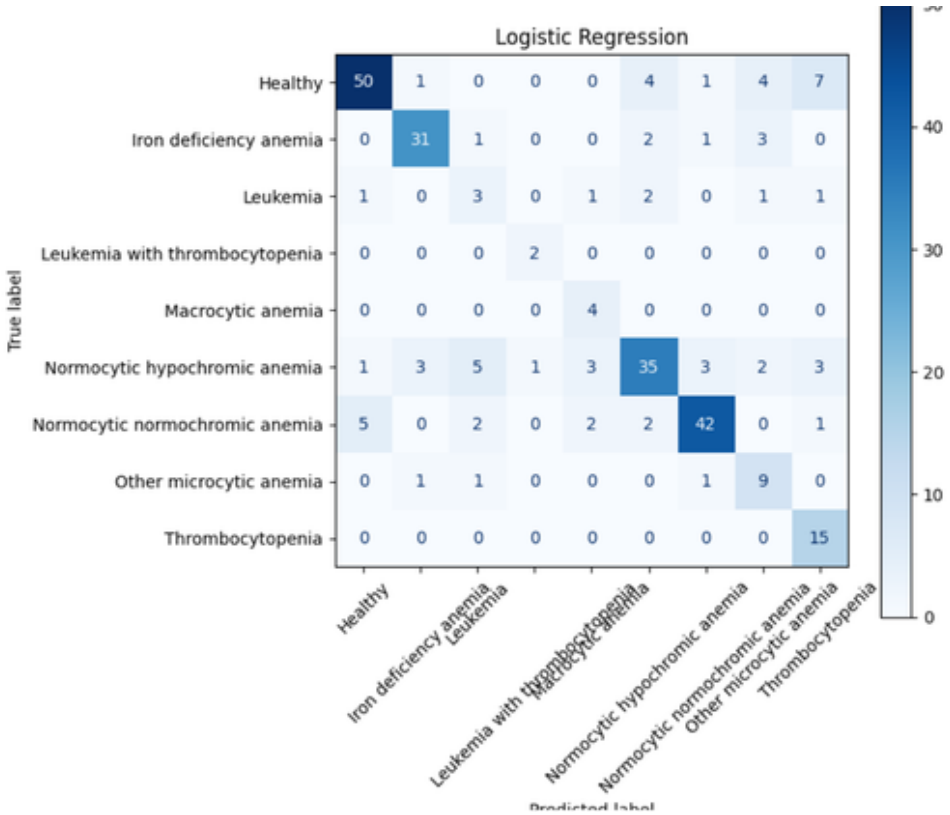


Figure 5.9: Confusion matrix of the Logistic Regression model

Relevant Metrics Calculated from the Confusion Matrix

To show how many diseased samples each model missed and how many healthy samples it wrongly flagged.

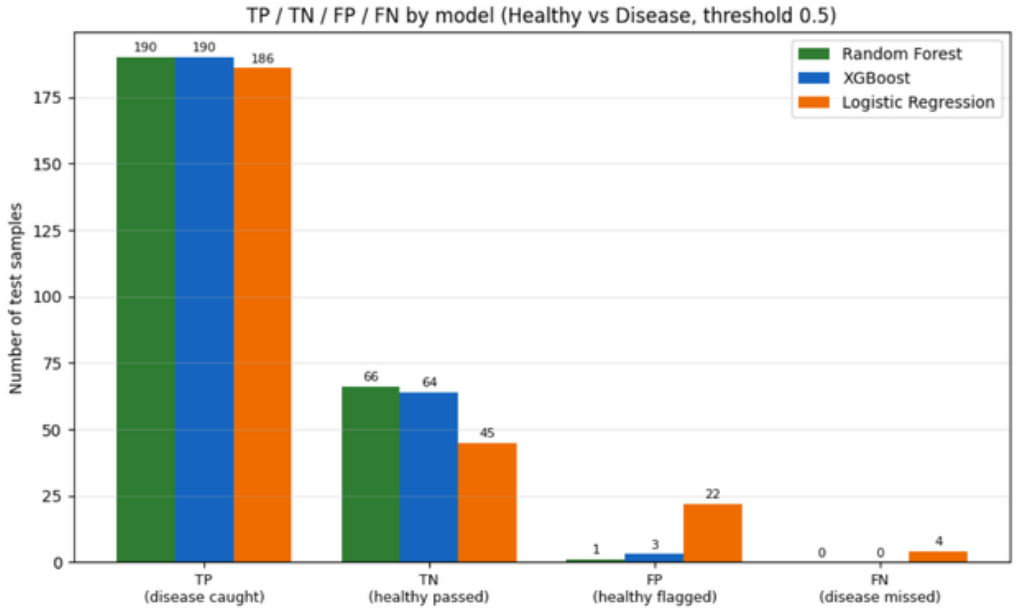


Figure 5.10: TP, TN, FP and FN counts by model (Healthy vs. Disease, threshold 0.5)

Model	TP	TN	FP	FN
Random Forest	190	66	1	0
XGBoost	190	64	3	0
Logistic Regression	186	45	22	4

Table 5.3: TP, TN, FP and FN counts by model (Healthy vs. Disease, threshold 0.5)

Per-class Performance:

Random Forest reached an F1-score of 0.94 or higher on eight of the nine classes, and its macro-averaged F1 (0.96) is close to its weighted F1 (0.99), which shows that its accuracy is not carried only by the large classes. Its weakest class was Macrocytic anemia (F1 = 0.80): all four test cases were found, but its precision of 0.67 means two other samples were wrongly assigned to it. The three errors in total were one Healthy, one Leukemia and one Other microcytic sample; two of them were predicted as Macrocytic anemia and one as Iron deficiency anemia (Figure 5.7).

Class	Precision	Recall	F1-score	Support
Healthy	1.00	0.99	0.99	67
Iron deficiency anemia	0.97	1.00	0.99	38
Leukemia	1.00	0.89	0.94	9
Leukemia with thrombocytopenia	1.00	1.00	1.00	2
Macrocytic anemia	0.67	1.00	0.80	4
Normocytic hypochromic anemia	1.00	1.00	1.00	56
Normocytic normochromic anemia	1.00	1.00	1.00	54
Other microcytic anemia	1.00	0.92	0.96	12
Thrombocytopenia	1.00	1.00	1.00	15
Macro average	0.96	0.98	0.96	257
Weighted average	0.99	0.99	0.99	257

Table 5.4: Per-class precision, recall and F1-score of the Random Forest model

		Random Forest		XGBoost		Logistic Reg.	
Class	n	Recall	F1	Recall	F1	Recall	F1
Healthy	67	0.99	0.99	0.96	0.98	0.75	0.81
Iron deficiency anemia	38	1.00	0.99	1.00	1.00	0.82	0.84
Leukemia	9	0.89	0.94	0.78	0.78	0.33	0.29
Leukemia with thrombocytopenia	2	1.00	1.00	0.00	0.00	1.00	0.80
Macrocytic anemia	4	1.00	0.80	1.00	0.80	1.00	0.57
Normocytic hypochromic anemia	56	1.00	1.00	1.00	0.99	0.62	0.69
Normocytic normochromic anemia	54	1.00	1.00	1.00	1.00	0.78	0.82
Other microcytic anemia	12	0.92	0.96	0.92	0.85	0.75	0.58
Thrombocytopenia	15	1.00	1.00	1.00	1.00	1.00	0.71
Macro average	257	0.98	0.96	0.85	0.82	0.78	0.68

Table 5.5: Per-class recall and F1-score of the three models

XGBoost matched Random Forest on the common classes but scored lower on Leukemia (F1 = 0.78) and Other microcytic anemia (F1 = 0.85), and it did not identify either of the two Leukemia

with thrombocytopenia test cases (F1 = 0.00), whereas Random Forest identified both. Its macro F1 (0.82) is therefore well below Random Forest (0.96). Logistic Regression had a recall of 1.00 on three rare classes but low precision on them (for example 0.40 for Macrocytic anemia and 0.56 for Thrombocytopenia), so it over-predicted these classes, probably because of the balanced training set, and it missed many Healthy and Normocytic hypochromic samples (recall 0.75 and 0.62). These class-level differences should be read with caution because the rarest classes have only 2 to 4 test samples.

Effect of the Decision Weight for Leukemia with Thrombocytopenia:

The deployed application multiplies the predicted probability of Leukemia with thrombocytopenia (LwT) by a decision weight of 2.0 before choosing the final class, whereas the results above use the unweighted Random Forest. To measure the effect of this adjustment, both rules were applied to the same 257 test records (Table 5.6).

Decision rule	Accuracy	LwT recall	LwT precision	Other classes predicted as LwT
Unweighted (W = 1.0)	98.83%	1.00	1.00	0
Weighted (W = 2.0)	98.83%	1.00	1.00	0

Table 5.6: Unweighted versus weighted decision rule on the test set

The weight of 2.0 did not change the prediction of any of the 257 test records. Random Forest already identified both LwT cases without the weight, so recall could not improve, and no other sample was pushed into the LwT class. The weight therefore had no measurable effect on this test set, in either direction. It is retained in the application as a precaution for borderline cases in which the LwT probability is close to that of another class, a situation that the two LwT test records do not represent. In a sensitivity check, raising the weight to 5.0 caused one other sample to be wrongly assigned to LwT (accuracy 98.44%), which shows that a larger weight trades precision for sensitivity.

The test set contains only 2 LwT records, so this comparison shows what the weight does on this test set; it does not establish the best weight or any clinical benefit, which would need a larger dataset.

Prediction Statistics on Unseen Data

Model	Total tested	Correct	Incorrect	Accuracy
Random Forest	257	254	3	98.83%
XGBoost	257	249	8	96.89%
Logistic Regression	257	191	66	74.32%

Table 5.7: Prediction statistics of the three models on the unseen test set

Feature Importance

Figure 5.11 shows the importance of each CBC parameter in the Random Forest model, measured as the mean decrease in Gini impurity across all trees.

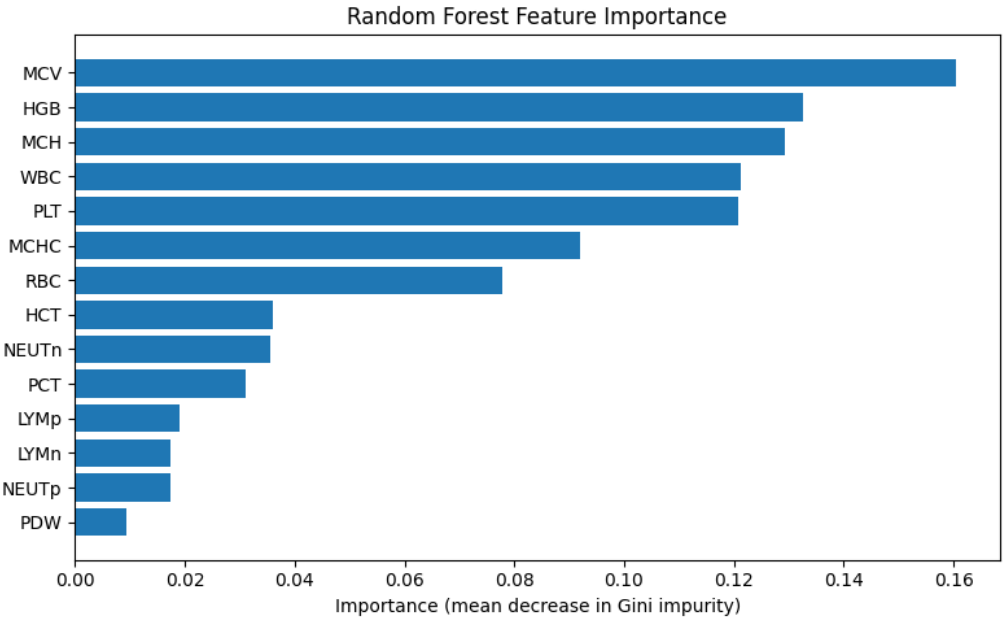


Figure 5.11: Random Forest feature importance of the 14 CBC parameters

MCV was the most important feature (0.160), followed by HGB (0.133) and MCH (0.129); together these three account for about 42% of the total importance. The red cell parameters (MCV, HGB, MCH, MCHC and RBC) make up about 59% of the total, which agrees with their role as the standard indicators for classifying anemia by red cell size, hemoglobin content and colour. WBC (0.121) and PLT (0.121) rank fourth and fifth, reflecting their importance for separating the Leukemia and Thrombocytopenia classes from the red cell anemias.

The lowest-ranked features are the differential white cell counts (LYMp, LYMn, NEUTp), PDW and, to a lesser degree, HCT, NEUTn and PCT. Several of these are the columns in which a single repeated value, equal to the column mean, fills about 61% of the records (Table 3.4), so they carry less real information, which may partly explain their low importance. HCT is also largely determined by RBC and MCV, so its importance may be shared with those features. Gini importance can be spread across correlated features and favours continuous variables, so the ranking shows which parameters the model relies on, not a causal effect of any parameter.

Discussion of Findings and Conclusions

Findings

- Ensemble models outperform the linear baseline. Random Forest (98.83%) and XGBoost (96.89%) far exceeded Logistic Regression (74.32%). CBC parameters interact in non-linear ways, for example in how HGB, MCV and MCH jointly separate the anemia sub-types, and tree ensembles model these interactions naturally.
- Random Forest is the most reliable model. It had the highest accuracy and weighted F1 and made the fewest errors (3 of 257 test samples). It identified both Leukemia with thrombocytopenia test cases, whereas XGBoost identified neither (F1 = 0.00), although with only 2 test samples this comparison rests on very few cases. Because of its overall performance, it was chosen for deployment in the Streamlit web application.
- Class balancing helped without inflating results. Applying SMOTE only to the training data let the models learn the minority classes (the training set was balanced at 269 per class), while the untouched test set keeps the evaluation honest.
- The important features are clinically meaningful. Hemoglobin (HGB), MCV and MCH were the highest-ranked features in the Random Forest importance analysis, with MCV first. These are standard indicators used to classify anemia by red cell size and hemoglobin content, which supports the model's validity.
- Consistency with the literature. The best results are in line with reported accuracies above 95% for tree-based and SVM models on CBC data in the reference papers (Farooq et al., 2025; Saputra et al., 2024; Rahman et al., 2024), although those studies differ in datasets and setup.

Conclusion: This project built and compared three machine learning models for classifying nine hematological conditions from CBC blood test data. Random Forest achieved the best results on unseen data (98.83% accuracy, 98.89% weighted F1) and was used as the prediction engine of the web application. XGBoost performed well (96.89%), while Logistic Regression (74.32%) confirmed that non-linear models are needed for this problem. The results show that routine

CBC parameters contain enough information for accurate automated screening of anemia and related blood disorders.

6 Model Deployment

6.1 Model Implementation

After evaluation, Random Forest was selected for deployment because it achieved the highest accuracy (98.83%) and weighted F1-score (98.89%) on the held-out test set. The trained model is wrapped in a web application built with Streamlit, a Python framework for building interactive data applications. The application loads the trained model from a single saved file, so it does not retrain when it starts.The listings below are abridged from the application source: imports and user-interface code are omitted, and the complete file is available in the project repository (Chapter 7).

Exporting the model: After training in Google Colab, the Random Forest pipeline and the information the application needs are saved together as one bundle using `joblib`.

```
import joblib, sklearn

FEATURES = list(X.columns) # the 14 CBC columns, in training order
classes = list(le.classes_) # the 9 class names (label order 0..8)
medians = X_train.median() # default value for each input field
counts = df['Diagnosis'].value_counts() # samples per class
n_rows = len(df) # number of rows in the dataset
LWT_CLASS = "Leukemia with thrombocytopenia"
W_LWT = 2.0 # decision weight (see below)

bundle = {
    "model": rf_model, # trained Random Forest (Chapter 4)
    "classes": classes,
    "medians": medians.to_dict(),
    "counts": counts.to_dict(),
    "n_rows": n_rows,
    "w_lwt": W_LWT,
    "features": FEATURES,
    "sklearn_version": sklearn.__version__,
}
joblib.dump(bundle, "anemia_model.joblib")
```

Application settings: The application uses the same 14 feature names and column order as the training notebook. Physiological limits for each parameter are used to reject impossible inputs.

```
MODEL_FILE = "anemia_model.joblib"
CLOSE_CALL_GAP = 0.15 # warn if the top-2 probabilities are closer than this
```

```
LWT_CLASS = "Leukemia with thrombocytopenia"

FEATURES = ["WBC", "LYMp", "NEUTp", "LYMn", "NEUTn", "RBC", "HGB",
            "HCT", "MCV", "MCH", "MCHC", "PLT", "PDW", "PCT"]

BOUNDS = { # (minimum, maximum) allowed for each input
    "WBC": (0.0, 30.0), "LYMp": (0.0, 100.0), "NEUTp": (0.0, 100.0),
    "LYMn": (0.0, 20.0), "NEUTn": (0.0, 25.0), "RBC": (0.0, 8.0),
    "HGB": (0.0, 25.0), "HCT": (0.0, 100.0), "MCV": (0.0, 150.0),
    "MCH": (0.0, 200.0), "MCHC": (0.0, 50.0), "PLT": (0.0, 1000.0),
    "PDW": (0.0, 30.0), "PCT": (0.0, 5.0),
}
```

Loading the model: The saved bundle is loaded from disk. If the file is missing, the application asks the user to upload it, and it stops if the feature list in the file does not match the application.

```
def load_bundle():
    if os.path.exists(MODEL_FILE):
        return joblib.load(MODEL_FILE)
    st.warning(f"{MODEL_FILE} not found next to this file. Upload it to continue.
        ")
    up = st.file_uploader("Upload anemia_model.joblib", type="joblib")
    if up is None:
        st.stop()
    return joblib.load(up)

bundle = load_bundle()
pipe = bundle["model"]
classes = list(bundle["classes"])
W_LWT = float(bundle["w_lwt"])

if bundle.get("features") != FEATURES:
    st.error("The model's feature list does not match this app.")
    st.stop()
```

Prediction: The `predict` function returns the predicted class, the class probabilities, and the raw model prediction. The probability of the rarest class, Leukemia with thrombocytopenia (LwT), is multiplied by a decision weight before the final class is chosen. This is a demonstration-time adjustment so that the rarest and most serious class is not ignored. It is applied only in the web application. The accuracy figures reported in Chapter 5 are for the unweighted Random Forest.

```
def predict(pipe, classes, w_lwt, row):
    proba = pipe.predict_proba(row)[0]
```

```
    raw_idx = int(np.argmax(proba))

    weighted = proba.copy()
    weighted[classes.index(LWT_CLASS)] *= w_lwt
    final_idx = int(np.argmax(weighted))

    probs = pd.Series(proba, index=classes).sort_values(ascending=False)
    return classes[final_idx], probs, final_idx != raw_idx, classes[raw_idx]
```

Safety checks: Two checks help users catch mistakes. First, each input field only accepts values inside the limits in BOUNDS. Second, hematocrit should be roughly RBC $\times$ MCV/10, so the application warns the user if the entered HCT differs from this by more than 15%. A "close call" warning is also shown when the two most probable classes differ by less than 0.15 in probability.

```
expected_hct = values["RBC"] * values["MCV"] / 10
if expected_hct > 0 and abs(values["HCT"] - expected_hct) / expected_hct > 0.15:
    st.warning("HCT differs from RBC x MCV / 10 by more than 15%. "
               "Please check the entered values for typing errors.")

gap = float(top3.iloc[0] - top3.iloc[1])
if gap < CLOSE_CALL_GAP:
    st.warning("Close call: the top two classes differ by only "
               f"{gap:.2f} in probability.")
```

Batch prediction: Uploaded files are processed row by row. Rows with missing, non-numeric or out-of-range values are skipped and a note is recorded, and the remaining rows are predicted.

```
def predict_table(X):
    X = X.apply(pd.to_numeric, errors="coerce")
    out_rows = []
    for idx, r in X.iterrows():
        note = ""
        if r.isna().any():
            note = "Missing or non-numeric value - skipped"
        else:
            bad = [f for f in FEATURES
                   if not (BOUNDS[f][0] <= r[f] <= BOUNDS[f][1])]
            if bad:
                note = "Outside allowed range: " + ", ".join(bad) + " - skipped"
        if note:
            out_rows.append({"Row": idx + 1, "Predicted class": None, "Note":
                note})
            continue
```

```
        label, probs, _, _ = predict(pipe, classes, W_LWT,
                                     pd.DataFrame([r])[FEATURES])
        out_rows.append({"Row": idx + 1, "Predicted class": label,
                         "Confidence": float(probs.iloc[0])})
    return pd.DataFrame(out_rows)
```

The complete source code of the application is available in the project repository (Chapter 7).

6.2 UI Design

The interface is organized into four tabs, so that a laboratory technician or clinician can use it without machine learning knowledge. The layout is generated with Streamlit's built-in components, with a small amount of custom CSS to enlarge the text.

- *Enter values:* a form with one numeric field for each of the 14 CBC parameters, arranged in three columns. Each field shows its unit, is limited to the allowed range, and starts with the dataset median as a default value. Pressing *Predict* shows the predicted class, a bar chart and table of the top three class probabilities, and a plain-language explanation of the predicted class.
- *Upload file:* the user uploads a CSV or Excel file with one row per patient and the 14 CBC columns (column names are not case-sensitive). A sample template can be downloaded. The application predicts every row, shows a results table, and allows the results to be downloaded as a CSV file. The user can also choose one row to see its full result and explanation.
- *Normal values:* a reference table with the unit, the normal range for men and for women, and a short description of what each of the 14 parameters measures.
- *Anemia types explained:* the full forms of the 14 parameters, and for each of the nine classes an explanation in simple words, the typical blood test pattern, common causes, and an easy way to remember it.

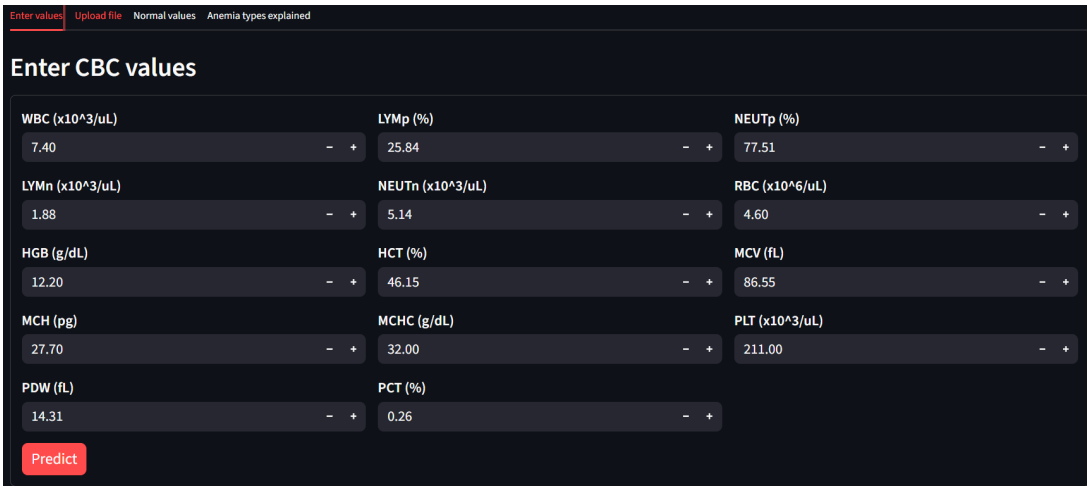


Figure 6.1: Input form for entering the 14 CBC values

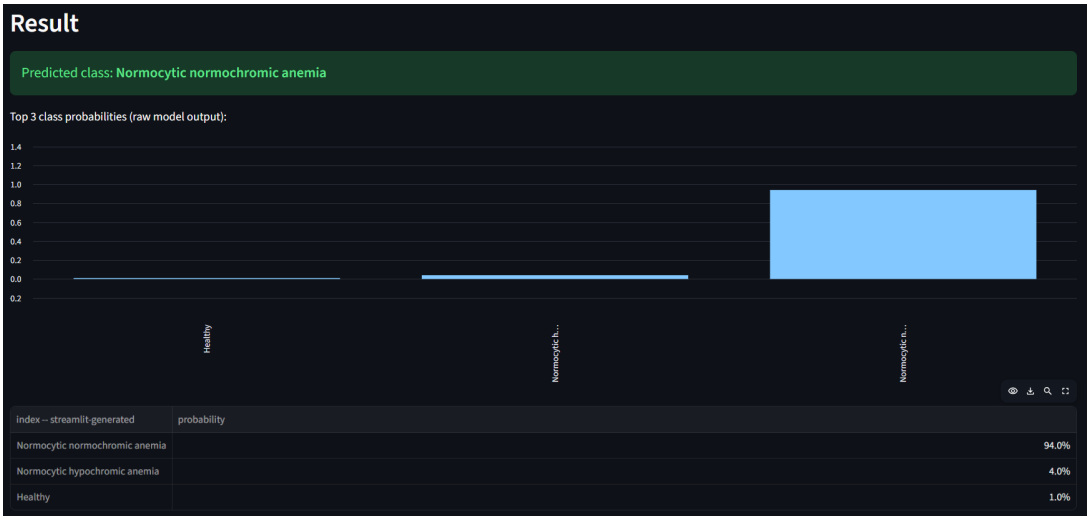


Figure 6.2: Prediction result with class probabilities and explanation

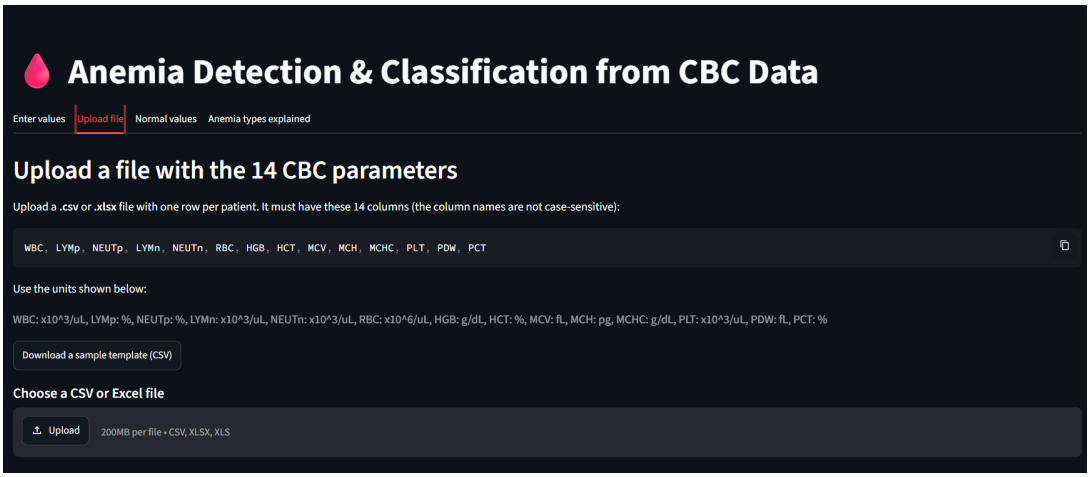


Figure 6.3: Batch prediction from an uploaded CSV or Excel file

7 Git History

Version control with Git was used throughout the project so that every stage of development, from the initial setup to the final application, is recorded and can be reproduced. The project is hosted on GitHub at `https://github.com/sulfathpm/anemia_detection`. Table 7.1 lists the commits in chronological order.

Date	Commit	Message
17-08-2026	b9a9533	Initial commit
25-09-2026	ce90b73	sprint updated
29-09-2026	e8771e4	updated sprint notebook
12-10-2026	d1a818f	add ui and requirements
13-10-2026	8f286bf	Add anemia prediction app and CBC dataset

Table 7.1: Git commit history of the project

The history shows the main phases of the work. The repository was created with the initial commit in August 2026. The model development notebook was then committed and updated in September 2026, covering the data analysis, preprocessing, model training and evaluation. In October 2026 the user interface and the `requirements.txt` file were added, followed by the final commit that added the anemia prediction application and the CBC dataset.

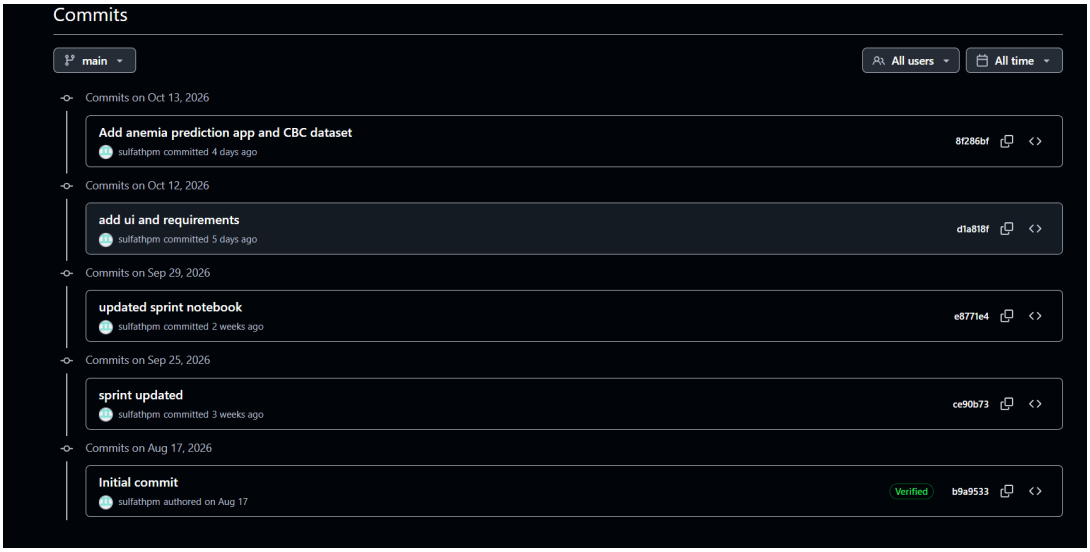


Figure 7.1: Commit history of the project on GitHub

The source code is available at the repository: `https://github.com/sulfathpm/anemia_detection`

8 Conclusions

This project developed an end-to-end machine learning system that classifies a patient's Complete Blood Count into one of nine categories: a healthy control class and eight anemia and related hematological subtypes. The system addresses a gap in the existing literature, where most studies treat anemia detection as a binary problem and few provide a deployable tool.

Three models were trained and compared on the same held-out test set. Random Forest achieved the best results, with 98.83% accuracy and a weighted F1-score of 98.89%, misclassifying only 3 of 257 test samples. XGBoost reached 96.89% accuracy but failed on the rarest class, Leukemia with thrombocytopenia, which has only 2 test samples, so this result should be interpreted with caution. Logistic Regression reached only 74.32%, which confirms that the class boundaries in CBC space are non-linear. Applying SMOTE only to the training partition allowed the models to learn the rare subtypes while keeping the evaluation free of data leakage.

The selected Random Forest model was implemented as a Streamlit web application that accepts the 14 standard CBC values and returns a predicted diagnosis in real time when run locally. The reported test results (98.83% accuracy) are for the unweighted Random Forest. The application additionally multiplies the predicted probability of Leukemia with thrombocytopenia by a decision weight before choosing the final class, so that this rare but serious condition is not overlooked. On the test set this adjustment changed none of the 257 predictions (Table 5.6), so the reported accuracy applies to the deployed rule as well; its value for borderline LwT cases remains untested because the test set has only 2 such records. Because it uses only open-source tools and runs on standard CPU hardware, the system is technically and economically feasible as a low-cost screening aid. It is intended to support, not replace, clinical judgment. The results also carry limitations: the dataset is small (1,281 records), the rarest classes have very few test samples (for example 2 for Leukemia with thrombocytopenia), and duplicate and out-of-range records were not removed before the split. These should be addressed before any clinical use.

9 Future Scope

The present system can be extended in the following directions:

- Validate the model on larger, independent and multi-centre clinical datasets, so that the performance on rare subtypes can be measured reliably.
- Remove duplicate records and apply the range-based capping of out-of-range values before splitting the data, and report the effect on the results.
- Use stratified k-fold cross-validation and hyperparameter tuning (for example grid or randomized search) to obtain more stable performance estimates than a single 80/20 split.
- Add explainability using methods such as SHAP or LIME, so that clinicians can see which CBC parameters drove each prediction.
- Add automatic extraction of CBC values from uploaded PDF or image reports using OCR, to remove manual data entry.
- Calibrate prediction probabilities and tune decision thresholds to favour recall for the serious conditions such as leukemia.
- Deploy the application on a cloud platform and extend it with patient record storage, report generation and a mobile-friendly interface.
- Evaluate the system with haematologists and laboratory staff in a pilot study before any real-world use.

10 Appendix

10.1 Minimum Software Requirements

Component	Requirement
Operating system	Windows 10 or later, Linux or macOS
Programming language	Python 3.9 or later
Data handling	Pandas, NumPy
Machine learning	Scikit-learn, XGBoost, imbalanced-learn
Visualization	Matplotlib, Seaborn
Web framework	Streamlit with CSS for the front end
Model storage	Joblib
Development environment	Google Colaboratory (training), VS Code or any code editor
Web browser	Any modern browser (Chrome, Edge, Firefox)
Version control	Git

Table 10.1: Minimum software requirements

10.2 Minimum Hardware Requirements

Component	Requirement
Processor	Dual-core CPU, 2 GHz or higher
Memory (RAM)	4 GB (8 GB recommended)
Storage	1 GB of free disk space
GPU	Not required
Internet	Required only for downloading libraries and for cloud notebooks

Table 10.2: Minimum hardware requirements

11 References

[1] M. S. Farooq, M. Hassan, G. Muhammad, O. Ali, Z. Zeeshan, M. Saleem, M. Ahmad, S. Abbas, M. A. Khan, and T. M. Ghazal, "Developing a Transparent Anaemia Prediction Model Empowered With Explainable Artificial Intelligence," *IEEE Access*, vol. 13, pp. 1307–1318, 2025.

[2] D. C. E. Saputra, K. Sunat, and T. Ratnaningsih, "SMOTE-MRS: A Novel SMOTE-Multiresolution Sampling Technique for Imbalanced Distribution to Improve Prediction of Anemia," *IEEE Access*, vol. 12, pp. 154675–154699, 2024.

[3] M. M. Rahman, M. U. Mojumdar, H. A. Shifa, N. R. Chakraborty, S. N. P. Stenin, and M. A. Hasan, "Anemia Disease Prediction Using Machine Learning Techniques and Performance Analysis," in *Proc. 2024 11th Int. Conf. Computing for Sustainable Global Development (INDIACom)*, 2024, pp. 1276–1282.

[4] E. Aboelnaga, "Anemia Types Classification," Kaggle dataset. [Online]. Available: https://www.kaggle.com/datasets/ehababoelnaga/anemia-types

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[6] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[8] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[9] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning," *Journal of Machine Learning Research*, vol. 18, no. 17, pp. 1–5, 2017.